

FLOWS - Flood Likelihood from pre-compiled Watershed Structure - Documentation

Thiago Ricardo Borges

September 26, 2026

Contents

1	First thoughts	3
2	Initial steps / handling geoTIF files	4
3	Finding basins	9
4	Analysing basins and handling infiltration	35
5	Applying results for the entire DEM	57

1 First thoughts

Flood-risk estimation can require the integration of several different types of data, including terrain, soil properties, land cover, precipitation, etc. This can become particularly challenging for intense, short-duration rainfall events - where conditions may change quickly and useful estimates may need to be produced within a limited time window.

FLAWS starts from a simple question: can the computationally expensive analysis of terrain topology and storage capacity be precomputed beforehand, so that flood-risk estimation can be generated quickly as soon as new rainfall data becomes available, while accepting some loss in physical fidelity?

The goal is not necessarily to replace advanced hydrodynamic simulation, but to provide a practical first-order estimate of flood-risk

The core idea: Water naturally accumulates in topographic depressions. Rather than attempting to deal with the full complexity of a hydrodynamic model, FLAWS precomputes the terrain structure and its potential storage relationships, then combines this information with rainfall-derived runoff estimates. Under an intentionally conservative scenario of high antecedent moisture, the model uses AMC III soil conditions to estimate water accumulation and depth distribution. This preprocessing works by finding bowl-like depressions and their contributing areas, where runoff tends to accumulate, and computes:

- Area
- Elevation - Volume relationship
- Geographical positioning
- Composite Curve Number (we will see more about this when handling infiltration)

Once this data is compiled, rainfall forecasts can be converted into water volumes and handled by our pre-calculated values — shifting most of the computational cost from runtime to a one-time preprocessing stage.

Our proposed approach looks something like this:

- Obtain a Digital Elevation Model (DEM) with an acceptable resolution
- Identify topographic bowl-like depressions (here, we will just call them "basins")
- Calculate storage - volume relationships for each basin
- Incorporate rainfall forecasts and infiltration estimates
- Estimate water accumulations and flood risk via the already calculated basin properties.

- Test on real-life events.

The goal is to investigate whether a precompiled representation of terrain storage can provide useful flood-risk estimates while remaining lightweight and quick enough to run on everyday laptops — making flood-awareness tools more accessible to small municipalities, schools, researchers, and citizen initiatives.

2 Initial steps / handling geoTIF files

First of all, we got to check out the first item in our list: obtaining a DEM with acceptable resolution. Now this is where things can get tricky: some countries have decent, high-quality DEM's made with LiDAR and other fancy methods that will provide us with extremely accurate information. But for the big majority (including me), those same resources are not easily accessible (if they can at all be found). In those cases, you would probably be best off with "open for public" elevation models such as Copernicus GLO-30 (that is very good, considering its freely available), but take in consideration that it may suffer from vegetation bias, and other inaccuracies that come with initiatives like this. Now, in my (the author) scenario, we are kind of lucky, since we have ANADEM.

ANADEM is a DEM brought to us by the Brazilian National Waters and Basic Sanitation Agency (ANA) in collaboration with the Federal University of Rio Grande do Sul (UFRGS). It is a refined version of Copernicus GLO-30, with reduced vegetation bias and developed with hydrological analysis in mind, available for South America.

Before continuing, it's important to remind the reader to be aware of DEM origins and quality, since those factors can and will affect the outcome of a model. It's also crucial to mention that, even though the goal of this documentation is to provide instructions and notes of FLOWS's construction to allow replication, the reader should have in mind that everything we use here was specifically made for Brazilian territory, so before adapting this project to your own needs please use parameters and resources for your own specific region/country.

Also let's have in mind that, for this project, hardware specs are so far:

- Lenovo Thinkpad E470
- Intel Core i5-7200U
- 8GB of DDR4 RAM
- 120GB SATA Solid State Drive
- Fedora Workstation 44

We need, as this project advances, to have this specs in mind.

Now, let's get to the really interesting stuff. So far, all that we want to do is find all the topographic depressions (let's call them basins), and get from them all the important data, like volume to water height relationship (probably the most important one so far). Later on, we will also analyze other factors like water infiltration and model accuracy.

First things first, ANADEM is provided as several .tif tiles that, together, form the whole DEM, as a single big file could easily be 60GB in size or more (We are talking of South America here - I wasn't able to find a DEM focused on hydrological analysis for Brazilian territory alone) . So, to get started, we'll make a simple python script (we'll be working with this language for the rest of the guide since libraries for our specific use are widely available, thus making our whole journey easier) that will read a single tile and output it in a visual format:

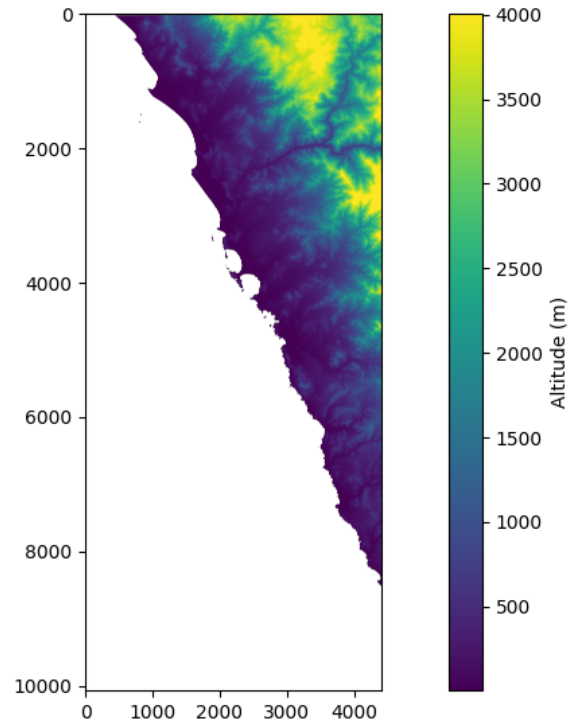
```
1 import rasterio
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # This code tests reading a .tif file, and generates a cool
6   image with it!
7
8 # We are here using a .tif file named mdt.tif.
9
10 with rasterio.open("mdt.tif") as src:
11     print("Bounds:", src.bounds)
12     print("CRS:", src.crs)
13     print("NoData:", src.nodata)
14     dem = src.read(1)
15
16 dem = np.where(dem == -9999, np.nan, dem)
17
18 valid = dem[~np.isnan(dem)]
19
20 print("Min:", np.min(valid))
21 print("Max:", np.max(valid))
22 print("Mean:", np.mean(valid))
23
24 plt.figure(figsize=(12,8))
25 plt.imshow(
26     dem,
27     vmin=np.percentile(valid, 2),
28     vmax=np.percentile(valid, 98))
```

```

28 )
29 plt.colorbar(label="Altitude (m)")
30 plt.show()
31

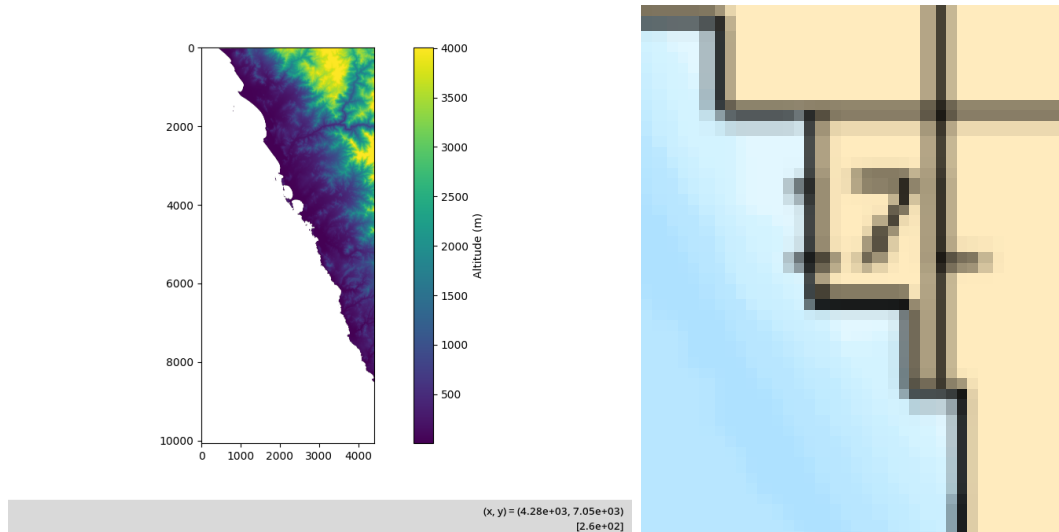
```

This code will generate us an output of this format:



(x, y) = (4.28e+03, 7.05e+03)
[2.6e+02]

Now please take in consideration that this specific tile had some noData zones - areas of the tile with no specific information about them, so they appear in blank. This should not be important for us now, since this tile in specific is located on the west coast of South America, and a visual inspection gives us that this noData zones are actually the ocean. Therefore, we will not worry about them here in this part of the path.



Now, we have to think how to actually take all of the info we've downloaded and turn it into an actually usable set of data. Some ideas would be:

- Load everything into a script and just find the basins → Well, not so fast there partner. As we've seen earlier, everything together will render us 60GB of information, all of that loaded at the same time, requiring a lot of storage and RAM. Since we don't have hardware like that available, we'll have to improvise.
- Load each individual tile one at a time → In theory this idea is pretty good! We would just need to load each downloaded tile individually and then find all the basins that are present in it. But there is a pretty important caveat: This solution is a victim to a widely discussed thing in hydrology and topography called Boundary Limitations → what would happen if a basin is cut in half by two adjacent tiles? This would generate inaccuracies that will be costly later. Therefore, not a totally viable solution.
- Load a central chunk of space and expand the borders if needed → How would this work: Basically we start to look for basins in an individual chunk in an individual tile that we will call the core box. We will try and look for all basins inside that area of space, and if any basin touches the boundaries delimited by the core box, we will expand our search space by adding a buffer to the sides. Then, we will look for basins that still belong to the original core box, but instead check whether any one touches the new boundaries delimited by the core box + buffer to the sides. If success, we get a file with a known area that does not suffer boundary limitations, but if we fail, we increase the buffer and try again. We repeat this process until we get a success.

This last idea seemed pretty good in theory and seems like the most straightforward option considering our scenario, but there are still some problems to it:

1. Terrain processing → Finding basins may seem straightforward, but actually applying all the things we learned into code, while having to mess with different file types and concepts that no one without a geography major would ever know (author included) is a completely different history.
2. How do we decide which basins should belong to the initial core box, and how do we deal with basins that belong (in area) to two different core boxes?
3. Way to big loops → What happens if a basin is just so big and we increase the buffer so much that the computer runs out of RAM?

But after some time brainstorming this topic, we are able to come up with a solution to each of these problems:

1. WhiteboxTools → Developed in Rust, WhiteboxTools is an advanced geospatial analysis library designed for hydrological processing. It comes with a range of tools, some made specifically for our objective of finding bowl-like depressions. Also, it can be used as a Python library! But not everything is perfect, and WhiteboxTools will still try to load everything we put at it on RAM without second thoughts. This will still break our computer if we try to load the entire DEM.

What are the system requirements?

The answer to this question depends strongly on the type of analysis and data that you intend to process. However, generally we find performance to be optimal with a recommended minimum of 8-16GB of memory (RAM), a modern multi-core processor (e.g. 64-bit i5 or i7), and an solid-state-drive (SSD). It is likely that *WhiteboxTools* will have satisfactory performance on lower-spec systems if smaller datasets are being processed. Because *WhiteboxTools* reads entire raster datasets into system memory (for optimal performance, and in recognition that modern systems have increasingly larger amounts of fast RAM), this tends to be the limiting factor for the upper-end of data size successfully processed by the library. 64-bit operating systems are recommended and extensive testing

https://jblindsay.github.io/wbt_book/print.html

Page 633 of 667

2. Geometry! → Every geometric shape has a unique point called the centroid (in some cases also known as a center of mass). What we can do is check, for each basin with a centroid belonging inside the core box boundaries, whether it touches the borders of the bigger core box + buffer boundaries. And since the centroid is unique, we can guarantee that each basin will belong in a single core box.
3. A little bit of math (and hope) → if we sum all the tile sizes, we get that our whole DEM, for the entire South America, weights around 63GB, Following that logic, the biggest floodable plain also in South America (Pantanal) would weight

around 0.9GB in size if it were its own big basin. So in theory, and big emphasis in theory, we should be able to run this program if we slice the big DEM in smaller individual core boxes the right way and optimize resource usage on our computer. But there is only one way to find out!

But you may still be wondering to yourself: "Okay, but how will we make our code look at all of our downloaded tiles and treat it as one big individual DEM without exceeding RAM and storage?" - for that question, we will also introduce the concept of a VRT (Virtual Raster).

A VRT is a lightweight XML-based file created by the GDAL ecosystem that acts essentially as a virtual mosaic. Instead of holding raw pixel data, it stores metadata and instructions on how multiple individual downloaded tiles should be aligned and organized to appear as a single, continuous, massive image. It basically works as follows (in very simple terms):

- Zero Space Duplication: We don't need to physically merge all tiles into a 60GB monster on disk. The VRT dynamically points to where each original tile is saved.
- On-Demand Reading: When the code starts processing the topography, it reads through the VRT, managing block reading and streaming only the necessary pixels directly from the disk at that exact moment.
- Seamless Global View: To the algorithm, the VRT makes the entire dataset look like a single continuous file, bypassing memory bottlenecks and removing the need to manage tile transitions manually.

So with all of that in mind, let's get started.

3 Finding basins

First of all, we have to create our VRT. There are several methods out there, but here it was chosen to use the terminal tool. To install it, you must follow instructions for your specific system. In this guide, we are on Fedora Workstation 44:

```
sudo dnf update && sudo dnf install gdal
```

After that, we must go to the folder containing all of our downloaded .tif files, and run the following command:

```
gdalbuildvrt <output file>.vrt *.tif
```

In this guide, we've chosen to name the output file as simply output.vrt, therefore we ran:

```
gdalbuildvrt output.vrt *.tif
```

```
untitledmacaw@fedora:~/Downloads$ ls
anadem_mgrs.zip  anadem_v1_18M.tif  anadem_v1_20F.tif  anadem_v1_21K.tif  anadem_v1_23J.tif
anadem_tiles.png  anadem_v1_18P.tif  anadem_v1_20G.tif  anadem_v1_21L.tif  anadem_v1_23K.tif
anadem_v1_17L.tif  anadem_v1_19F.tif  anadem_v1_20H.tif  anadem_v1_21M.tif  anadem_v1_23L.tif
anadem_v1_17M.tif  anadem_v1_19G.tif  anadem_v1_20J.tif  anadem_v1_21N.tif  anadem_v1_23M.tif
anadem_v1_17N.tif  anadem_v1_19H.tif  anadem_v1_20K.tif  anadem_v1_21P.tif  anadem_v1_24K.tif
anadem_v1_18F.tif  anadem_v1_19J.tif  anadem_v1_20M.tif  anadem_v1_22H.tif  anadem_v1_24L.tif
anadem_v1_18G.tif  anadem_v1_19K.tif  anadem_v1_20N.tif  anadem_v1_22J.tif  anadem_v1_24M.tif
anadem_v1_18H.tif  anadem_v1_19L.tif  anadem_v1_20P.tif  anadem_v1_22K.tif  anadem_v1_25L.tif
anadem_v1_18K.tif  anadem_v1_19M.tif  anadem_v1_21F.tif  anadem_v1_22L.tif  anadem_v1_25M.tif
anadem_v1_18L.tif  anadem_v1_19N.tif  anadem_v1_21H.tif  anadem_v1_22M.tif  RJ_2016.pdf
anadem_v1_18M.tif  anadem_v1_19P.tif  anadem_v1_21J.tif  anadem_v1_22N.tif  Sistema-Brasileiro-de-Classificacao-de-Solos-2025.pdf
untitledmacaw@fedora:~/Downloads$ gdalbuildvrt output.vrt *.tif
0...10...20...30...40...50...60...70...80...90...100 - done.
untitledmacaw@fedora:~/Downloads$ ls
anadem_mgrs.zip  anadem_v1_18P.tif  anadem_v1_20H.tif  anadem_v1_21N.tif  anadem_v1_24K.tif
anadem_tiles.png  anadem_v1_19F.tif  anadem_v1_20J.tif  anadem_v1_21P.tif  anadem_v1_24L.tif
anadem_v1_17L.tif  anadem_v1_19G.tif  anadem_v1_20K.tif  anadem_v1_22H.tif  anadem_v1_24M.tif
anadem_v1_17M.tif  anadem_v1_19H.tif  anadem_v1_20M.tif  anadem_v1_22J.tif  anadem_v1_25L.tif
anadem_v1_17N.tif  anadem_v1_19J.tif  anadem_v1_20N.tif  anadem_v1_22K.tif  anadem_v1_25M.tif
anadem_v1_18F.tif  anadem_v1_19K.tif  anadem_v1_20P.tif  anadem_v1_22L.tif  output.vrt
anadem_v1_18G.tif  anadem_v1_19L.tif  anadem_v1_21F.tif  anadem_v1_22M.tif  RJ_2016.pdf
anadem_v1_18H.tif  anadem_v1_19M.tif  anadem_v1_21H.tif  anadem_v1_22N.tif  Sistema-Brasileiro-de-Classificacao-de-Solos-2025.pdf
anadem_v1_18K.tif  anadem_v1_19N.tif  anadem_v1_21J.tif  anadem_v1_23J.tif
anadem_v1_18L.tif  anadem_v1_19P.tif  anadem_v1_21K.tif  anadem_v1_23K.tif
anadem_v1_18M.tif  anadem_v1_20F.tif  anadem_v1_21L.tif  anadem_v1_23L.tif
anadem_v1_18N.tif  anadem_v1_20G.tif  anadem_v1_21M.tif  anadem_v1_23M.tif
untitledmacaw@fedora:~/Downloads$
```

The next thing we should do is also install Whitebox:

```
pip install whitebox
```

```
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ pip install whitebox
Collecting whitebox
  Downloading whitebox-2.3.6-py2.py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: Click>=6.0 in ./venv/lib64/python3.14/site-packages (from whitebox) (8.4.1)
Downloading whitebox-2.3.6-py2.py3-none-any.whl (74 kB)
Installing collected packages: whitebox
Successfully installed whitebox-2.3.6

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ pip freeze > requirements.txt
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$
```

Perfect! So now let's move on to the next step. Our current code workflow can be described as follows:

1. Chose 4 values (min x, min y, max x, max y) that will delimitate our core box
2. Run a loop where we:
 - Increase the buffer by a fixed value
 - Convert the piece from the original VRT delimited by our core box + buffer from referencing system EPSG:4326 (degrees) to referencing system EPSG:5800 (metres). This is important because calculations later on involving degrees will gives us inaccuracies.

- Check if the current buffer is way too big (we will set a buffer limit for that)
- Find all bowl-like depressions + their contribution areas and, for each one, find its centroid
- Filter results where the centroid belongs to the original core box
- For those results, we check if any one touches the border of the core box + buffer area
- If yes, we run another cycle. If not, we break the loop

3. We save our results and analyse each individual basin.

To process the topography and find bowl-like depressions, we will rely on a sequence of functions provided by WhiteboxTools:

- **breach_single_cell_pits**: Resolves single-cell topographic pits in the raw DEM via selective breaching rather than filling. This ensures continuous flow routing while preventing the over-smoothing of surrounding terrain.
- **d8_pointer**: Calculates flow directions using the standard D8 algorithm, assigning each grid cell an integer pointer toward its steepest downslope neighbor.
- **sink**: Identifies local depressions and pour points across the DEM where surface runoff accumulates and cannot drain further.
- **watershed**: Delineates the full contributing catchment area (basin) for each pour point identified by **sink** using the D8 pointer grid.
- **depth_in_sink**: Computes the maximum storage depth for each basin grid cell by measuring the elevation difference between the cell and its basin's spill elevation.

After catchment boundaries are calculated, we use NumPy array masking to isolate basins assigned to the current core box and strip away unwanted terrain without messing with basin geometry:

1. Centroid Classification: Using `scipy.ndimage.center_of_mass`, we calculate the spatial centroid (x, y) of each basin raster in pixel coordinates and project them to ground coordinates in meters (EPSG:5880). If a centroid falls strictly within the core tile bounds ($c_{\min x} \leq x \leq c_{\max x}$ and $c_{\min y} \leq y \leq c_{\max y}$), its basin ID is stored in the set `owned_by_core_ids`.
2. Boolean Mask Generation (`np.isin`): We convert `owned_by_core_ids` to a 1D NumPy array (`core_ids_array`) and evaluate `np.isin(basins_data, core_ids_array)`. This generates a 2D boolean grid (`mask`) where pixels belonging to target basins are set to `True`, and adjacent basins leaking into the buffer are set to `False`.

3. Raster Filtering (`np.where`): Passing the boolean mask to `np.where(mask, basins_data, nodata_val)` replaces all `False` pixels with `nodata_val` (or 0). The identical mask is applied to the depth output (`filtered_depth = np.where(mask, depth_data, nodata_val_depth)`). This cleanly clips away all non-core basins while maintaining the intact vector geometry of every basin originating inside the core tile.

With that in mind, let's start translating that to code. We are able to initially come up with this:

```
1 import os
2 import geopandas as gpd
3 from shapely.geometry import box
4 from osgeo import gdal
5 from whitebox import WhiteboxTools
6 import rasterio
7 import numpy as np
8 import scipy.ndimage as ndimage
9 import matplotlib.pyplot as plt
10 import matplotlib.patches as patches
11 from matplotlib.colors import ListedColormap
12
13 gdal.UseExceptions()
14
15 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
16     """
17     This will cut the DEM according to topography, preserving
18     basin boundaries (AKA no basins get cut)
19     Criteria for cutting: All basins with a centroid that
20     belongs to core tile A will be grouped together on the same
21     TIF
22
23     :param path_vrt: Path to our .vrt file
24     :param out_dir: Path for our output
25     :param core_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
26     :param tile_id: Unique tile ID
27
28     """
29
30     os.makedirs(out_dir, exist_ok=True)
31     minx, miny, maxx, maxy = core_bounds_deg
32
33     # Projects Core Tile limit from 4326 (degrees) to 5800 (
34     # meters)
```

```

31     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
maxx, maxy)]}, crs="EPSG:4326")
32     df_core_5800 = df_core.to_crs("EPSG:5880")
33     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
maxx, maxy) em metros
34
35     # Dynamic buffer initial setup
36     buffer_current = 0.1
37     buffer_steps = 0.1
38     buffer_limit = 4
39     sucess = False
40
41     wbt = WhiteboxTools()
42     wbt.set_working_dir(os.path.abspath(out_dir))
43     wbt.verbose = False
44
45     while not sucess and buffer_current <= buffer_limit:
46         print(f"[{tile_id}] Trying to process with a margin of {
buffer_current}")
47
48         # Define limits
49         buf_minx = minx - buffer_current
50         buf_miny = miny - buffer_current
51         buf_maxx = maxx + buffer_current
52         buf_maxy = maxy + buffer_current
53
54         # Names of temporary files
55         filename_dem = f"temp_{tile_id}_dem.tif"
56         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
57         filename_d8 = f"temp_{tile_id}_d8.tif"
58         filename_sinks = f"temp_{tile_id}_sinks.tif"
59         filename_basins = f"basins_final_{tile_id}.tif"
60         filename_depth = f"depth_final_{tile_id}.tif"
61
62         path_dem = os.path.join(out_dir, filename_dem)
63         path_basins = os.path.join(out_dir, filename_basins)
64
65         try:
66             # Cuts OG VRT
67             gdal.Warp(
68                 destNameOrDestDS=path_dem,
69                 srcDSOrSrcDSTab=path_vrt,
70                 format='GTiff',
71                 outputBounds=(buf_minx, buf_miny, buf_maxx,

```

```

buf_maxy),
72         outputBoundsSRS='EPSG:4326',
73         dstSRS='EPSG:5880',
74         xRes=30, yRes=30,
75         creationOptions=['COMPRESS=DEFLATE', 'TILED=YES'
]
76     )
77
78     # WhiteboxTools pipeline here
79     print(f"        -> Running WhiteboxTools ...")
80     wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)
81     wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
82     wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
83     wbt.watershed(d8_ptr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
84
85     # Collision Test pipeline here
86
87     print(f"        -> Running collision test ...")
88
89     with rasterio.open(path_basins) as src:
90         basins_data = src.read(1)
91         transform = src.transform
92
93     # a) Grabs IDS from all basins that touch the borders
of the current matrix (core box + current buffer)
94     top_edge = basins_data[0, :]
95     bottom_edge = basins_data[-1, :]
96     left_edge = basins_data[:, 0]
97     right_edge = basins_data[:, -1]
98
99     edge_ids = set(np.unique(np.concatenate([top_edge,
bottom_edge, left_edge, right_edge])))
100     edge_ids.discard(0) # Removes noData
101
102     # b) Calculates the centroid of each basin present
inside the raster file
103     unique_ids = np.unique(basins_data)
104     unique_ids = unique_ids[unique_ids > 0] # Removes
noData (I hope)
105

```

```

1106         centers = ndimage.center_of_mass(basins_data > 0,
1107         basins_data, unique_ids)
1108
1109         owned_by_core_ids = set()
1110         c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
1111
1112         # c) Filters which basins have their centroid INSIDE
1113         core tile
1114         for basin_id, (row_center, col_center) in zip(
1115         unique_ids, centers):
1116             # Converts pixel (line, column) to geo cords (x,
1117             y in meters)
1118             x_coord, y_coord = transform * (col_center,
1119             row_center)
1120
1121             if (c_minx <= x_coord <= c_maxx) and (c_miny <=
1122             y_coord <= c_maxy):
1123                 owned_by_core_ids.add(basin_id)
1124
1125         # d) Is there any basin with centroid inside core
1126         tile that leaks outside of the buffer?
1127         leaking_basins = owned_by_core_ids.intersection(
1128         edge_ids)
1129
1130         if len(leaking_basins) > 0:
1131             print(f"[FAIL] {len(leaking_basins)} basin(s)
1132             leaked outside current buffer")
1133             buffer_current += buffer_steps
1134
1135         else:
1136             print(f"[SUCESS] All interesting basins fit
1137             within a buffer of {buffer_current}")
1138
1139         wbt.depth_in_sink(dem=filename_dem_clean, output
1140         =filename_depth, zero_background=True)
1141
1142         sucess = True
1143
1144     except Exception as e:
1145         print(f"[ERROR]: {e}")
1146         print(f"    -> Ending runtime ...")
1147         break

```

```

139
140     # File cleanup
141     temp_files = [filename_dem, filename_dem_clean,
142 filename_d8, filename_sinks]
143     if not sucess:
144         temp_files.extend([filename_basins, filename_depth])
145     # Deletes if failed
146
147     for f in temp_files:
148         path_f = os.path.join(out_dir, f)
149         if os.path.exists(path_f):
150             os.remove(path_f)
151
152     if not sucess and buffer_current >= buffer_limit:
153         print(f"[WARNING]: Buffer limit reached. Consider
154 revising terrain.")
155
156 if True:
157     PATH_VRT = '~/Downloads/output.vrt'
158     OUT_DIR = 'test_basins_dynamic'
159     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
160
161     dynamic_cut(
162         path_vrt=PATH_VRT,
163         out_dir=OUT_DIR,
164         core_bounds_deg=BOUNDS_TESTING,
165         tile_id="test_01"
166     )

```

Here, we chose a small box just to run a quick check (we will keep using this same area until we're okay to start handling our entire dataset). Our output looked something like this:



```

[test_01] Trying to process with a margin of 0.1
-> Running WhiteboxTools ...
-> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
-> Running WhiteboxTools ...
-> Running colision test ...
[SUCCESS] All interesting basins fit within a buffer of 0.2

```

test_basins_dynamic

- basins_final_test_01.tif U
- depth_final_test_01.tif U

But how can we actually verify that our code is indeed working? Wouldn't it be cool if we could somehow get a visual output of basins and out core box and core box

+ buffer at the end of runtime?

For that to work, we have to adapt our code a little bit:

```
1 import os
2 import geopandas as gpd
3 from shapely.geometry import box
4 from osgeo import gdal
5 from whitebox import WhiteboxTools
6 import rasterio
7 import numpy as np
8 import scipy.ndimage as ndimage
9 import matplotlib.pyplot as plt
10 import matplotlib.patches as patches
11 from matplotlib.colors import ListedColormap
12
13 gdal.UseExceptions()
14
15 def plot_tile_diagnostics(
16     basins_data,
17     transform,
18     core_bounds_5800,
19     buf_deg_bounds,
20     centers,
21     unique_ids,
22     owned_by_core_ids,
23     tile_id,
24 ):
25     """
26     Takes our results from the dynamic cutting and outputs them
27     as a visual .png file so we can have visual
28     proof that all centroids are contained inside the core box
29     It is advised to read the dynamic_cut function before
30     reading this one (down below).
31
32     :param basins_data: Our basins raster array
33     :param transform: Our transformation matrix (provided by
34     rasterio)
35     :param core_bounds_5800: Tuple (minx, miny, maxx, maxy) that
36     defines our core box limits
37     :param buf_deg_bounds: Tuple (buf_minx, buf_miny, buf_maxx,
38     buf_maxy) that defines our core box + buffer limits
39     :param centers: List with all coordinates (line and column)
40     from each of the basins centroids
41     :param unique_ids: Array containing all of the unique basin
```

```

IDs present in the image (removing 0)
36 :param tile_id: Our tile ID
37
38 """
39
40 # WARNING: DO NOT REMOVE FIG.
41 fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
42
43 # 1. Generating unique colors for each ID
44 # Generates random colors for each ID to try to avoid visual
45 # problems with neighbor basins
46 max_id = int(basins_data.max())
47 np.random.seed(
48     42
49 )
50 color_array = np.random.rand(max_id + 1, 4)
51 color_array[
52     0
53 ] = [ # Defines background color (ID 0 = noData)
54     0,
55     0,
56     0,
57     0,
58 ]
59 custom_cmap = ListedColormap(color_array)
60
61 # 2. Plots basins raster with custom colors
62 im = ax.imshow(
63     basins_data,
64     cmap=custom_cmap,
65     interpolation='nearest', # Avoids blurring
66     extent=[
67         transform[2],
68         transform[2] + transform[0] * basins_data.shape[1],
69         transform[5] + transform[4] * basins_data.shape[0],
70         transform[5],
71     ],
72 )
73
74 buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
75 df_buf = gpd.GeoDataFrame(
76     {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy)
77 ]}],
78     crs='EPSG:4326',

```

```

77     )
78     b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
total_bounds
79     c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
80
81     # Draws Core Box limits
82     core_rect = patches.Rectangle(
83         (c_minx, c_miny),
84         c_maxx - c_minx,
85         c_maxy - c_miny,
86         linewidth=2.5,
87         edgecolor='red',
88         facecolor='none',
89         linestyle='--',
90         label='Core Tile',
91     )
92     ax.add_patch(core_rect)
93
94     # Draws Core Box + Buffer tile
95     buf_rect = patches.Rectangle(
96         (b_minx, b_miny),
97         b_maxx - b_minx,
98         b_maxy - b_miny,
99         linewidth=2,
100        edgecolor='blue',
101        facecolor='none',
102        label='Buffer Tile (Final)',
103    )
104    ax.add_patch(buf_rect)
105
106    # Plots centroids
107    first_core = True
108    for basin_id, (row_center, col_center) in zip(unique_ids,
centers):
109        x_coord, y_coord = transform * (col_center, row_center)
110
111        if basin_id in owned_by_core_ids:
112            label_text = 'Centroid Core' if first_core else ''
113            ax.plot(
114                x_coord,
115                y_coord,
116                marker='*',
117                color='black',
118                markersize=5,

```

```

119         markeredgecolor='white',
120         markeredgewidth=0.5,
121         label=label_text,
122     )
123     first_core = False
124
125     else:
126         ax.plot(
127             x_coord,
128             y_coord,
129             marker='o',
130             color='dimgray',
131             markersize=2,
132             alpha=0.4,
133         )
134
135     # Adjust zoom
136     ax.set_xlim(b_minx, b_maxx)
137     ax.set_ylim(b_miny, b_maxy)
138
139     # Graphic aesthetics
140     ax.set_facecolor('#f4f4f4')
141     ax.grid(True, linestyle=':', alpha=0.5, color='gray')
142     plt.title(
143         f'Basins check - Tile: {tile_id}',
144         fontsize=14,
145         fontweight='bold',
146     )
147     plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
148     plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
149     plt.legend(loc='upper right', frameon=True, facecolor='white', shadow=True)
150
151     plt.tight_layout()
152     plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
153     plt.close()
154     print(
155         f'        -> Image stored as: diagnostic_{tile_id}.png'
156     )
157
158 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
159     """
160     This will cut the DEM according to topography, preserving
161     basin boundaries (AKA no basins get cut)

```

```

161     Criteria for cutting: All basins with a centroid that
162     belongs to core tile A will be grouped together on the same
163     TIF
164
165     :param path_vrt: Path to our .vrt file
166     :param out_dir: Path for our output
167     :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
168     :param tile_id: Unique tile ID
169
170     """
171
172     os.makedirs(out_dir, exist_ok=True)
173     minx, miny, maxx, maxy = core_bounds_deg
174
175     # Projects Core Tile limist from 4326 (degress) to 5800 (
176     # meters)
177     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
178     maxx, maxy)]}, crs="EPSG:4326")
179     df_core_5800 = df_core.to_crs("EPSG:5880")
180     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
181     maxx, maxy) em metros
182
183     # Dynamic buffer initial setup
184     buffer_current = 0.1
185     buffer_steps = 0.1
186     buffer_limit = 4
187     sucess = False
188
189     wbt = WhiteboxTools()
190     wbt.set_working_dir(os.path.abspath(out_dir))
191     wbt.verbose = False
192
193     while not sucess and buffer_current <= buffer_limit:
194         print(f"[{tile_id}] Trying to process with a margin of {
195         buffer_current}")
196
197         # Define limits
198         buf_minx = minx - buffer_current
199         buf_miny = miny - buffer_current
200         buf_maxx = maxx + buffer_current
201         buf_maxy = maxy + buffer_current
202
203         # Names of temporary files
204         filename_dem = f"temp_{tile_id}_dem.tif"

```

```

199     filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
200     filename_d8 = f"temp_{tile_id}_d8.tif"
201     filename_sinks = f"temp_{tile_id}_sinks.tif"
202     filename_basins = f"basins_final_{tile_id}.tif"
203     filename_depth = f"depth_final_{tile_id}.tif"
204
205     path_dem = os.path.join(out_dir, filename_dem)
206     path_basins = os.path.join(out_dir, filename_basins)
207
208     try:
209         # Cuts OG VRT
210         gdal.Warp(
211             destNameOrDestDS=path_dem,
212             srcDSOrSrcDSTab=path_vrt,
213             format='GTiff',
214             outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
215             outputBoundsSRS='EPSG:4326',
216             dstSRS='EPSG:5880',
217             xRes=30, yRes=30,
218             creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]
219         )
220
221         # WhiteboxTools pipeline here
222         print(f"        -> Running WhiteboxTools ...")
223         wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)
224         wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
225         wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
226         wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
227
228         # Colision Test pipeline here
229
230         print(f"        -> Running colision test ...")
231
232         with rasterio.open(path_basins) as src:
233             basins_data = src.read(1)
234             transform = src.transform
235
236         # a) Grabs IDS from all basins that touch the bordes

```

```

of the current matrix (core box + current buffer)
237     top_edge = basins_data[0, :]
238     bottom_edge = basins_data[-1, :]
239     left_edge = basins_data[:, 0]
240     right_edge = basins_data[:, -1]
241
242     edge_ids = set(np.unique(np.concatenate([top_edge,
bottom_edge, left_edge, right_edge])))
243     edge_ids.discard(0) # Removes noData
244
245     # b) Calculates the centroid of each basin present
inside the raster file
246     unique_ids = np.unique(basins_data)
247     unique_ids = unique_ids[unique_ids > 0] # Removes
noData (I hope)
248
249     centers = ndimage.center_of_mass(basins_data > 0,
basins_data, unique_ids)
250
251     owned_by_core_ids = set()
252     c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
253
254     # c) Filters which basins have their centroid INSIDE
core tile
255     for basin_id, (row_center, col_center) in zip(
unique_ids, centers):
256         # Converts pixel (line, column) to geo cords (x,
y in meters)
257         x_coord, y_coord = transform * (col_center,
row_center)
258
259         if (c_minx <= x_coord <= c_maxx) and (c_miny <=
y_coord <= c_maxy):
260             owned_by_core_ids.add(basin_id)
261
262     # d) Is there any basin with centroid inside core
tile that leaks outside of the buffer?
263     leaking_basins = owned_by_core_ids.intersection(
edge_ids)
264
265     if len(leaking_basins) > 0:
266         print(f"[FAIL] {len(leaking_basins)} basin(s)
leaked outside current buffer")
267         buffer_current += buffer_steps

```

```

268
269         else:
270             print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
271
272             wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
273
274             # testing quick visuals
275             print(f"        -> Generating image ...")
276             buf_bounds = (buf_minx, buf_miny, buf_maxx,
buf_maxy)
277             plot_tile_diagnostics(
278                 basins_data,
279                 transform,
280                 core_bounds_5800,
281                 buf_bounds,
282                 centers,
283                 unique_ids,
284                 owned_by_core_ids,
285                 tile_id,
286             )
287             sucess = True
288
289
290
291     except Exception as e:
292         print(f"[ERROR]: {e}")
293         print(f"        -> Ending runtime ...")
294         break
295
296     #
297     temp_files = [filename_dem, filename_dem_clean,
filename_d8, filename_sinks]
298     if not sucess:
299         temp_files.extend([filename_basins, filename_depth])
300     # Deletes if failed
301
302     for f in temp_files:
303         path_f = os.path.join(out_dir, f)
304         if os.path.exists(path_f):
305             os.remove(path_f)
306
307     if not sucess and buffer_current >= buffer_limit:

```

```

307         print(f"[WARNING]: Buffer limit reached. Consider
           revising terrain.")
308
309 if True:
310     PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
311     OUT_DIR = 'test_basins_dynamic'
312     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
313
314     dynamic_cut(
315         path_vrt=PATH_VRT,
316         out_dir=OUT_DIR,
317         core_bounds_deg=BOUNDS_TESTING,
318         tile_id="test_01"
319     )
320
321

```

What this code does is basically, at the end of the loop, if we get a success, we call a function to generate a visual graph of our results, and output it as a .png file.

```

[test_01] Trying to process with a margin of 0.1
-> Running WhiteboxTools ...
-> Running collision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
-> Running WhiteboxTools ...
-> Running collision test ...
[SUCCESS] All interesting basins fit within a buffer of 0.2
-> Generating image ...
-> Image stored as: diagnostic_test_01.png

```

But before we check result yet, we have to exclude the basins that are outside our area of interest.

To remove those other basins, we adapt the code some more:

```

1 import os
2 import geopandas as gpd
3 from shapely.geometry import box
4 from osgeo import gdal
5 from whitebox import WhiteboxTools
6 import rasterio
7 import numpy as np
8 import scipy.ndimage as ndimage
9 import matplotlib.pyplot as plt
10 import matplotlib.patches as patches
11 from matplotlib.colors import ListedColormap

```

```

12
13 gdal.UseExceptions()
14
15 def plot_tile_diagnostics(
16     basins_data,
17     transform,
18     core_bounds_5800,
19     buf_deg_bounds,
20     centers,
21     unique_ids,
22     owned_by_core_ids,
23     tile_id,
24 ):
25     """
26     Takes our results from the dynamic cutting and outputs them
27     as a visual .png file so we can have visual
28     proof that all centroids are contained inside the core box
29     It is advised to read the dynamic_cut function before
30     reading this one (down below).
31
32     :param basins_data: Our basins raster array
33     :param transform: Our transformation matrix (provided by
34     rasterio)
35     :param core_bounds_5800: Tuple (minx, miny, maxx, maxy) that
36     defines our core box limits
37     :param buf_deg_bounds: Tuple (buf_minx, buf_miny, buf_maxx,
38     buf_maxy) that defines our core box + buffer limits
39     :param centers: List with all coordinates (line and column)
40     from each of the basins centroids
41     :param unique_ids: Array containing all of the unique basin
42     IDs present in the image (removing 0)
43     :param tile_id: Our tile ID
44
45     """
46
47     # WARNING: DO NOT REMOVE FIG.
48     fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
49
50     # 1. Generating unique colors for each ID
51     # Generates random colors for each ID to try to avoid visual
52     # problems with neighbor basins
53     max_id = int(basins_data.max())
54     np.random.seed(
55         42
56     )

```

```

48     )
49     color_array = np.random.rand(max_id + 1, 4)
50     color_array[
51         0
52     ] = [ # Defines background color (ID 0 = noData)
53         0,
54         0,
55         0,
56         0,
57     ]
58     custom_cmap = ListedColormap(color_array)
59
60     # 2. Plots basins raster with custom colors
61     im = ax.imshow(
62         basins_data,
63         cmap=custom_cmap,
64         interpolation='nearest', # Avoids blurring
65         extent=[
66             transform[2],
67             transform[2] + transform[0] * basins_data.shape[1],
68             transform[5] + transform[4] * basins_data.shape[0],
69             transform[5],
70         ],
71     )
72
73     buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
74     df_buf = gpd.GeoDataFrame(
75         {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy)
76         ]}],
77         crs='EPSG:4326',
78     )
79     b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
total_bounds
80
81     # Draws Core Box limits
82     core_rect = patches.Rectangle(
83         (c_minx, c_miny),
84         c_maxx - c_minx,
85         c_maxy - c_miny,
86         linewidth=2.5,
87         edgecolor='red',
88         facecolor='none',
89         linestyle='--',

```

```

90         label='Core Tile',
91     )
92     ax.add_patch(core_rect)
93
94     # Draws Core Box + Buffer tile
95     buf_rect = patches.Rectangle(
96         (b_minx, b_miny),
97         b_maxx - b_minx,
98         b_maxy - b_miny,
99         linewidth=2,
100        edgecolor='blue',
101        facecolor='none',
102        label='Buffer Tile (Final)',
103    )
104    ax.add_patch(buf_rect)
105
106    # Plots centroids
107    first_core = True
108    for basin_id, (row_center, col_center) in zip(unique_ids,
109        centers):
110
111        x_coord, y_coord = transform * (col_center, row_center)
112
113        if basin_id in owned_by_core_ids:
114            label_text = 'Centroid Core' if first_core else ''
115            ax.plot(
116                x_coord,
117                y_coord,
118                marker='',
119                color='black',
120                markersize=5,
121                markeredgecolor='white',
122                markeredgewidth=0.5,
123                label=label_text,
124            )
125            first_core = False
126
127        else:
128            ax.plot(
129                x_coord,
130                y_coord,
131                marker='',
132                color='dimgray',
133                markersize=2,
134                alpha=0.4,

```

```

133         )
134
135     # Adjust zoom
136     ax.set_xlim(b_minx, b_maxx)
137     ax.set_ylim(b_miny, b_maxy)
138
139     # Graphic aesthetics
140     ax.set_facecolor('#f4f4f4')
141     ax.grid(True, linestyle=':', alpha=0.5, color='gray')
142     plt.title(
143         f'Basins check - Tile: {tile_id}',
144         fontsize=14,
145         fontweight='bold',
146     )
147     plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
148     plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
149     plt.legend(loc='upper right', frameon=True, facecolor='white', shadow=True)
150
151     plt.tight_layout()
152     plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
153     plt.close()
154     print(
155         f'        -> Image stored as: diagnostic_{tile_id}.png'
156     )
157
158 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
159     """
160     This will cut the DEM according to topography, preserving
161     basin boundaries (AKA no basins get cut)
162     Criteria for cutting: All basins with a centroid that
163     belongs to core tile A will be grouped together on the same
164     TIF
165
166     :param path_vrt: Path to our .vrt file
167     :param out_dir: Path for our output
168     :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
169     :param tile_id: Unique tile ID
170
171     """
172     os.makedirs(out_dir, exist_ok=True)
173     minx, miny, maxx, maxy = core_bounds_deg

```

```

173     # Projects Core Tile limist from 4326 (degress) to 5800 (
174     meters)
175     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
176     maxx, maxy)]}, crs="EPSG:4326")
177     df_core_5800 = df_core.to_crs("EPSG:5880")
178     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
179     maxx, maxy) em metros
180
181     # Dynamic buffer initial setup
182     buffer_current = 0.1
183     buffer_steps = 0.1
184     buffer_limit = 4
185     sucess = False
186
187     wbt = WhiteboxTools()
188     wbt.set_working_dir(os.path.abspath(out_dir))
189     wbt.verbose = False
190
191     while not sucess and buffer_current <= buffer_limit:
192         print(f"[{tile_id}] Trying to process with a margin of {
193         buffer_current}")
194
195         # Define limits
196         buf_minx = minx - buffer_current
197         buf_miny = miny - buffer_current
198         buf_maxx = maxx + buffer_current
199         buf_maxy = maxy + buffer_current
200
201         # Names of temporary files
202         filename_dem = f"temp_{tile_id}_dem.tif"
203         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
204         filename_d8 = f"temp_{tile_id}_d8.tif"
205         filename_sinks = f"temp_{tile_id}_sinks.tif"
206         filename_basins = f"basins_final_{tile_id}.tif"
207         filename_depth = f"depth_final_{tile_id}.tif"
208
209         path_dem = os.path.join(out_dir, filename_dem)
210         path_basins = os.path.join(out_dir, filename_basins)
211
212         try:
213             # Cuts OG VRT
214             gdal.Warp(
215                 destNameOrDestDS=path_dem,
216                 srcDSOrSrcDSTab=path_vrt,

```

```

213         format='GTiff',
214         outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
215         outputBoundsSRS='EPSG:4326',
216         dstSRS='EPSG:5880',
217         xRes=30, yRes=30,
218         creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]
219     )
220
221     # WhiteboxTools pipeline here
222     print(f"        -> Running WhiteboxTools ...")
223     wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)
224     wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
225     wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
226     wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
227
228     # Colision Test pipeline here
229
230     print(f"        -> Running colision test ...")
231
232     with rasterio.open(path_basins) as src:
233         basins_data = src.read(1)
234         transform = src.transform
235
236     # a) Grabs IDS from all basins that touch the bordes
of the current matrix (core box + current buffer)
237     top_edge = basins_data[0, :]
238     bottom_edge = basins_data[-1, :]
239     left_edge = basins_data[:, 0]
240     right_edge = basins_data[:, -1]
241
242     edge_ids = set(np.unique(np.concatenate([top_edge,
bottom_edge, left_edge, right_edge])))
243     edge_ids.discard(0) # Removes noData
244
245     # b) Calculates the centroid of each basin present
inside the raster file
246     unique_ids = np.unique(basins_data)
247     unique_ids = unique_ids[unique_ids > 0] # Removes

```

```

noData (I hope)
248
249         centers = ndimage.center_of_mass(basins_data > 0,
basins_data, unique_ids)
250
251         owned_by_core_ids = set()
252         c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
253
254         # c) Filters which basins have their centroid INSIDE
core tile
255         for basin_id, (row_center, col_center) in zip(
unique_ids, centers):
256             # Converts pixel (line, column) to geo cords (x,
y in meters)
257             x_coord, y_coord = transform * (col_center,
row_center)
258
259             if (c_minx <= x_coord <= c_maxx) and (c_miny <=
y_coord <= c_maxy):
260                 owned_by_core_ids.add(basin_id)
261
262         # d) Is there any basin with centroid inside core
tile that leaks outside of the buffer?
263         leaking_basins = owned_by_core_ids.intersection(
edge_ids)
264
265         if len(leaking_basins) > 0:
266             print(f"[FAIL] {len(leaking_basins)} basin(s)
leaked outside current buffer")
267             buffer_current += buffer_steps
268
269         else:
270             print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
271
272         wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
273
274         # Removing excess basins
275         print(f"    -> Applying mask to keep only basins
of interest ...")
276
277         # a) Transforms the valid IDs set into a numpy
array

```

```

278         core_ids_array = np.array(list(owned_by_core_ids
279 ))
280
281         # b) Creates a boolean mask (True if the pixel
282         belongs to a basin of interest)
283         mask = np.isin(basins_data, core_ids_array)
284
285         # c) Checks which noData value is used by DEM
286         and applies mask over basin_data
287         with rasterio.open(path_basins, 'r+') as dst:
288             lines, columns = basins_data.shape
289             area_m_square = (lines * 30) * (columns *
290 30)
291
292             print(f"        -> Total area processed: {
293 area_m_square / 1000000:.2f} km2")
294             nodata_val = dst.nodata if dst.nodata is not
295 None else 0
296
297             filtered_basins = np.where(mask, basins_data
298 , nodata_val)
299             dst.write(filtered_basins, 1)
300
301         # d) Open, filters and overwrites depth TIF
302         path_depth = os.path.join(out_dir,
303 filename_depth)
304         with rasterio.open(path_depth, 'r+') as dst:
305             depth_data = dst.read(1)
306             nodata_val_depth = dst.nodata if dst.nodata
307 is not None else 0
308
309             filtered_depth = np.where(mask, depth_data,
310 nodata_val_depth)
311             dst.write(filtered_depth, 1)
312
313         # testing quick visuals
314         print(f"        -> Generating image ...")
315         buf_bounds = (buf_minx, buf_miny, buf_maxx,
316 buf_maxy)
317
318         plot_tile_diagnostics(
319             filtered_basins,
320             transform,
321             core_bounds_5800,
322             buf_bounds,
323             centers,

```

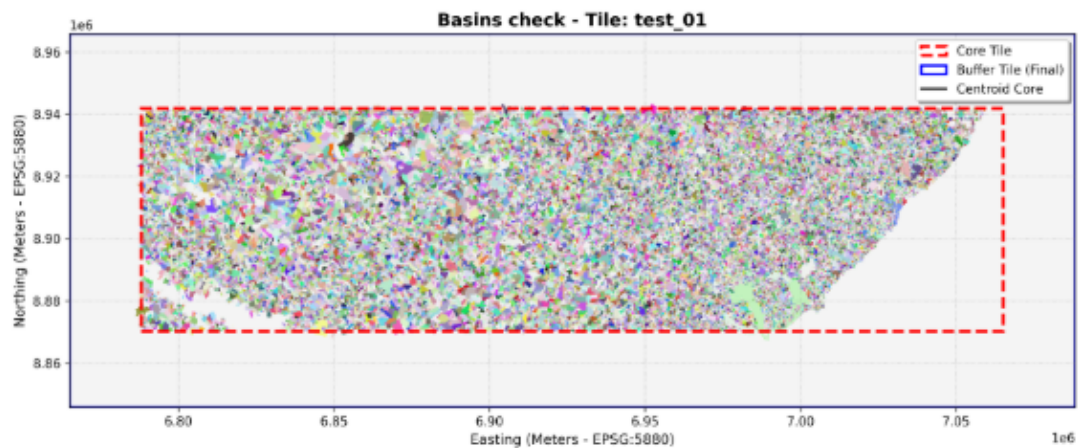
```

311         unique_ids,
312         owned_by_core_ids,
313         tile_id,
314     )
315     sucess = True
316
317
318
319     except Exception as e:
320         print(f"[ERROR]: {e}")
321         print(f"    -> Ending runtime ...")
322         break
323
324     #
325     temp_files = [filename_dem, filename_dem_clean,
326 filename_d8, filename_sinks]
327     if not sucess:
328         temp_files.extend([filename_basins, filename_depth])
329     # Deletes if failed
330
331     for f in temp_files:
332         path_f = os.path.join(out_dir, f)
333         if os.path.exists(path_f):
334             os.remove(path_f)
335
336     if not sucess and buffer_current >= buffer_limit:
337         print(f"[WARNING]: Buffer limit reached. Consider
338 revising terrain.")
339
340 if True:
341     PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
342     OUT_DIR = 'test_basins_dynamic'
343     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
344
345     dynamic_cut(
346         path_vrt=PATH_VRT,
347         out_dir=OUT_DIR,
348         core_bounds_deg=BOUNDS_TESTING,
349         tile_id="test_01"
350     )

```

This will give us our final output for this section:

```
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$ python3 terrain_cut_centroid.py
[test_01] Trying to process with a margin of 0.1
-> Running WhiteboxTools ...
-> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
-> Running WhiteboxTools ...
-> Running colision test ...
[SUCCESS] All interesting basins fit within a buffer of 0.2
-> Applying mask to keep only basins of interest ...
-> Total area processed: 38796.10 km2
-> Generating image ...
-> Image stored as: diagnostic_test_01.png
(.venv) untitledmacaw@fedora:~/Desktop/Alagamentos/FloodGraph$
```



As we can see, we no longer have those extra basins on our final file!

4 Analysing basins and handling infiltration

So now we have it: we can actually find basins! But how do we analyse each one of them?

Before we get anything started, we have to address the elephant in the room: rain water infiltration. As you may already guess, soil is permeable → this means that water will only start overflowing when terrain gets too saturated. Not only that, but different terrains will behave in different ways when rainwater falls, where some will soak a lot of volume but others will basically be impermeable. So this raises a question: How can we deal with this problem?

This rushes us to present another very cool thing in hydrology: the Curve Number (CN) method. It was developed by the US Department of Agriculture (USDA) Natural Resources Conservation Service (NRCS) (back when it was formerly known as the Soil

Conservation Service (SCS)). The SCS CN method estimates precipitation excess as a function of the cumulative precipitation depth, soil cover, land use, and antecedent soil moisture as:

$$P_e = \begin{cases} 0, & \text{for } P < I_a \\ \frac{(P - I_a)^2}{P - I_a + S}, & \text{otherwise} \end{cases} \quad (1)$$

where P_e = accumulated precipitation excess at time t ; P = accumulated rainfall depth at time t ; I_a = the initial abstraction (initial loss); and S = potential maximum retention, a measure of the ability of a watershed to abstract and retain storm precipitation. Until the accumulated rainfall exceeds the initial abstraction, the precipitation excess, hence the runoff, will be zero. From analysis of results from many small experimental watersheds, the SCS developed an empirical relation of I_a and S :

$$I_a = 0.2 * S \quad (2)$$

Therefore, the cumulative excess as time t is:

$$P_e = \frac{(P - 0.2 * S)^2}{(P + 0.8 * S)} \quad (3)$$

Incremental excess for a time interval is computed as the difference between the accumulated excess at the end of and beginning of the period. The maximum retention, S , and watershed characteristics are related through an intermediate parameter, the curve number (commonly abbreviated CN) as:

$$S = \begin{cases} \frac{1000 - 10 * CN}{CN}, & \text{foot / pound system} \\ \frac{25400 - 254 * CN}{CN}, & \text{The obviously better alternative, also known as SI} \end{cases} \quad (4)$$

Jokes aside, CN values range from 100 (for water bodies) to approximately 30 for permeable soils with high infiltration rates.

The curve number that is entered should be a "composite" curve number that represents all of the different soil groups and land use combinations in the subbasin. Typically, curve numbers are derived from soil maps. I_a defines the amount of precipitation that must fall before excess precipitation results. Here, we will use (2) (as provided by the curve number provider that we will mention shortly).

For a watershed that consists of several soil types and land uses, a composite CN

is calculated as:

$$CN_{composite} = \frac{\sum A_i CN_i}{\sum A_i} \quad (5)$$

In which $CN_{composite}$ = the composite CN used for runoff volume computations; i = and index of watersheds subdivisions of uniform land use and soil type; CN_i = the CN subdivisions i ; and A_i = the drainage area of subdivision i .

One important thing to mention though is that the standard SCS CN soil classification method (developed initially with US soils) has been found to be inaccurate when used with more tropical soils like the ones in Brazil. More about this topic, and also a reclassification of soil groups for Brazilian soils and land uses can be found in Sartori et al. (2005).

This same paper was used by ANA to generate a raster file (.tif) of CN values for the entire country (not the whole South American terrain this time), so we are kind of lucky! But the reader should have in mind that the correct CN numbers must be used for use cases on different terrain, since it will deeply affect model outputs.

We need to have this ideas in mind, but we can leave some of those aside for now, since we'll now figure out how to analyse each individual basin. First, we have to look at our last program's outputs (in this case, the tif files):

- basins final → tells us where basins are.
- depth final → tells us the max depth of each of the basins pieces until it overflows.

Here, the secret is to use NumPy masks. We tell Python to isolate only the pixels belonging to a specific ID, and we run calculations with only those pixels. Then, we will convert the CN of every pixel from AMC (antecedent moisture condition) II, equal to normal moisture conditions, to AMC III (saturated soil). We'll do this so that the worst case scenario of a flood caused by a storm that came when the soil was already saturated from previous rain (that is a completely plausible and possible scenario) also gets considered in our calculations.

After all of that is done, we will use the adjusted CN values to calculate the surface runoff (the amount of water that doesn't soak into the ground). Because NumPy works with arrays, we can apply the SCS-CN formulas to all of our different storm scenarios at the same time. By multiplying this runoff by the basin's drainage area, we get the total volume of water. and storing all these results into a CSV table for easy reading.

Another interesting point to mention is how we calculate our composite CN. Let's return to Equation (5):

$$CN_{composite} = \frac{\sum A_i CN_i}{\sum A_i} \quad (6)$$

However, we have a key detail to notice: our CN data is stored in a raster file (.tif) where space is represented by a bunch of pixels.

Taking n as the number of distinct soil/land-use categories, A_{pixel} as the individual area of a single pixel, p_i as the number of pixels belonging to category i , and N_{total} as the total number of pixels in the watershed ($\sum_{i=1}^n p_i = N_{total}$), we can rewrite the expression as:

$$\begin{aligned}
 CN_{composite} &= \frac{\sum A_i CN_i}{\sum A_i} \\
 &= \frac{A_{pixel} \cdot p_1 CN_1 + \dots + A_{pixel} \cdot p_n CN_n}{A_{pixel} \cdot N_{total}} \\
 &= \frac{A_{pixel} \cdot (p_1 CN_1 + p_2 CN_2 + \dots + p_n CN_n)}{A_{pixel} \cdot N_{total}} \quad (7) \\
 &= \frac{\sum_{i=1}^n p_i CN_i}{N_{total}} \\
 &= \text{arithmetic mean of all individual pixel CN values}
 \end{aligned}$$

So, with all of that in mind, let's write a new function to our previous code:

```

1 def simulate_basin_volumes_worst_case(path_basins, path_depth,
2   path_cn, owned_ids, out_dir, tile_id):
3     """
4     Analyses individual basins, and for each one calculates:
5     -> Composite CN (after we convert each one from AMC II to
6     AMC III)
7     -> Volume / Water height ratio
8     -> Other important stuff
9
10    and then stores the values in a cute table!
11
12    :param path_basins: Path to our basins_final .tif file
13    :param path_depth: Path to our depth_final .tif file
14    :param path_cn: Path to our CN table .tif file (observation:
15    their CN values are AMC II standard. We must convert them
16    first)
17    :param owned_ids: Our owned_by_core_ids variable (go to the
18    main function if in doubt)
19    :param out_dir: Our output directory
20    :param tile_id: Our tile id (go to the main function if in
    doubt)
21
22    """
23
24    # saves different storm strengths to test (in mm)
25    # according to opticalscientific.com:

```

```

21     # light rain: < 60mm / 24hrs ( < 2.5mm / hour, and we
converted it to 24hrs)
22     # moderate rain: 60 to 180mm / 24hrs (2.5 to 7.5 mm / hour)
23     # heavy rain: 180mm to 1200mm / 24hrs (7.5 to 50mm / hour)
24     # Violent rain: > 1200 / 24hrs ( > 50mm / hour)
25     # here, we will add some in-between values for calculations'
sake
26     scenarios = np.array([60, 180, 300, 600, 800, 1200], dtype=
float)
27
28     all_dfs = []
29
30     with rasterio.open(path_basins) as src_b, \
31         rasterio.open(path_depth) as src_d, \
32         rasterio.open(path_cn) as src_cn:
33
34         # Validates dimentions (between our CN and DEM)
35         if not (src_b.shape == src_d.shape == src_cn.shape):
36             raise ValueError(
37                 f"[ALIGNMENT ERROR] Dimentions are not
compatible\n"
38                 f"         -> Basins: {src_b.shape}\n"
39                 f"         -> Depth:  {src_d.shape}\n"
40                 f"         -> CN:      {src_cn.shape}"
41             )
42
43         # 2. Validates resolution and geographical positioning (
Geotransform)
44         # Allows a tiny tiny tolerance for floating point Warp
imperfections
45         if not np.allclose(src_b.transform, src_cn.transform,
atol=1e-5):
46             raise ValueError(
47                 "[ALIGNMENT ERROR] The matrixes cover entirely
different areas or have different spacial resolution"
48             )
49
50         basins_data = src_b.read(1)
51         depth_data = src_d.read(1)
52         cn_data = src_cn.read(1)
53
54         pixel_width = src_b.transform[0]
55         pixel_height = -src_b.transform[4]
56         pixel_area = pixel_width * pixel_height

```

```

57
58     for basin_id in owned_ids:
59         mask = (basins_data == basin_id)
60         if not np.any(mask):
61             continue
62
63         basin_area = np.sum(mask) * pixel_area
64
65         valid_cn = cn_data[mask]
66         valid_cn = valid_cn[valid_cn > 0]
67         # Converts array to float to prevent uint8 overflow
68
69         if len(valid_cn) == 0:
70             print(f"        -> Skipping basin {basin_id}: No
valid CN data.")
71             continue
72
73         valid_cn_float = valid_cn.astype(float)
74
75         # converts entire matrix from AMC II to AMC III
pixel by pixel
76         valid_cn_iii = (23 * valid_cn_float) / (10 + 0.13 *
valid_cn_float)
77
78         # calculates composite already on the worst case
scenario
79         cn_iii = np.mean(valid_cn_iii)
80         cn_ii_mean_original = np.mean(valid_cn)
81
82         # SCS CN with saturated soil
83         S = (25400 / cn_iii) - 254
84         Q = np.zeros_like(scenarios)
85
86         valid_p = scenarios > (0.2 * S)
87         Q[valid_p] = ((scenarios[valid_p] - 0.2 * S)**2) / (
scenarios[valid_p] + 0.8 * S)
88
89         volume_rain_m3 = (Q / 1000) * basin_area
90
91         depths = depth_data[mask]
92         max_depth = depths.max()
93
94         h_range = np.linspace(0, max_depth, 50)
95         v_range = []

```

```

96
97         for h in h_range:
98             water_depth = np.maximum(0, h - (max_depth -
100 depths))
101             v_range.append(np.sum(water_depth) * pixel_area)
102
103         final_heights = np.interp(volume_rain_m3, v_range,
104 h_range)
105
106         df = pd.DataFrame({
107             'Basin_ID': int(basin_id),
108             'Precipitation_mm': scenarios,
109             'AMC_II_Original': np.round(cn_ii_mean_original,
110 2),
111             'AMC_III_Saturated': np.round(cn_iii, 2),
112             'Runoff_Q_mm': np.round(Q, 2),
113             'Volume_m3': np.round(volume_rain_m3, 2),
114             'Max_height_m': np.round(final_heights, 2)
115         })
116
117         all_dfs.append(df)
118
119     if all_dfs:
120         combined_df = pd.concat(all_dfs, ignore_index=True)
121         csv_path = os.path.join(out_dir, f'all_basins_{tile_id}
122 _wcs.csv')
123         combined_df.to_csv(csv_path, index=False)
124         print(f"    -> Exported worst-case volume table:
125 all_basins_{tile_id}_wcs.csv")

```

You may still be wondering: "Okay, but if our CN raster file is limited to only Brazil, while our DEM consists of a bigger area, what will happen when we access an area outside Brazil?". Well, in this code, pixels consisting of noData are isolated, and basins that are completely outside our studied terrain will fall on the valid $cn > 0$ check, and will be skipped. So, we will not worry about it.

But before continuing to implement this new function on the final code, we also have to deal with another thing: How will we know where the basin is in the real world if we only store volume related values and CN numbers? For that, we will also need to modify our cut function from the previous section. For keeping things simple,

we'll just store our spacial info by storing it as a GeoPackage, a vectorial and usually lightweight file format. With both our CSV table and our GeoPackage sharing a basin id collumn, we can later access the volume-related data on the space we need to work on.

With all of that in mind, let's convert that to code:

```
1 import os
2 import geopandas as gpd
3 from shapely.geometry import box, shape
4 from osgeo import gdal
5 from whitebox import WhiteboxTools
6 import rasterio
7 import rasterio.features
8 import numpy as np
9 import pandas as pd
10 import scipy.ndimage as ndimage
11 import matplotlib.pyplot as plt
12 import matplotlib.patches as patches
13 from matplotlib.colors import ListedColormap
14
15 gdal.UseExceptions()
16
17 def simulate_basin_volumes_worst_case(path_basins, path_depth,
18   path_cn, owned_ids, out_dir, tile_id):
19     """
20     Analyses individual basins, and for each one calculates:
21     -> Composite CN (after we convert each one from AMC II to
22     AMC III)
23     -> Volume / Water height ratio
24     -> Other important stuff
25
26     and then stores the values in a cute table!
27
28     :param path_basins: Path to our basins_final .tif file
29     :param path_depth: Path to our depth_final .tif file
30     :param path_cn: Path to our CN table .tif file (observation:
31     their CN values are AMC II standard. We must convert them
32     first)
33     :param owned_ids: Our owned_by_core_ids variable (go to the
34     main function if in doubt)
35     :param out_dir: Our output directory
36     :param tile_id: Our tile id (go to the main function if in
37     doubt)
```

```

33     """
34
35     # saves different storm strengths to test (in mm)
36     # according to opticalscientific.com:
37     # light rain: < 60mm / 24hrs ( < 2.5mm / hour, and we
converted it to 24hrs)
38     # moderate rain: 60 to 180mm / 24hrs (2.5 to 7.5 mm / hour)
39     # heavy rain: 180mm to 1200mm / 24hrs (7.5 to 50mm / hour)
40     # Violent rain: > 1200 / 24hrs ( > 50mm / hour)
41     # here, we will add some in-between values for calculations'
sake
42     scenarios = np.array([60, 180, 300, 600, 800, 1200], dtype=
float)
43
44     all_dfs = []
45
46     with rasterio.open(path_basins) as src_b, \
47         rasterio.open(path_depth) as src_d, \
48         rasterio.open(path_cn) as src_cn:
49
50         # Validates dimentions (between our CN and DEM)
51         if not (src_b.shape == src_d.shape == src_cn.shape):
52             raise ValueError(
53                 f"[ALIGNMENT ERROR] Dimentions are not
compatible\n"
54                 f"         -> Basins: {src_b.shape}\n"
55                 f"         -> Depth:  {src_d.shape}\n"
56                 f"         -> CN:      {src_cn.shape}"
57             )
58
59         # 2. Validates resolution and geographical positioning (
Geotransform)
60         # Allows a tiny tiny tolerance for floating point Warp
imperfections
61         if not np.allclose(src_b.transform, src_cn.transform,
atol=1e-5):
62             raise ValueError(
63                 "[ALIGNMENT ERROR] The matrixes cover entirely
different areas or have different spacial resolution"
64             )
65
66         basins_data = src_b.read(1)
67         depth_data = src_d.read(1)
68         cn_data = src_cn.read(1)

```

```

69
70     pixel_width = src_b.transform[0]
71     pixel_height = -src_b.transform[4]
72     pixel_area = pixel_width * pixel_height
73
74     for basin_id in owned_ids:
75         mask = (basins_data == basin_id)
76         if not np.any(mask):
77             continue
78
79         basin_area = np.sum(mask) * pixel_area
80
81         valid_cn = cn_data[mask]
82         valid_cn = valid_cn[valid_cn > 0]
83         # Converts array to float to prevent uint8 overflow
84
85         if len(valid_cn) == 0:
86             print(f"        -> Skipping basin {basin_id}: No
valid CN data.")
87             continue
88
89         valid_cn_float = valid_cn.astype(float)
90
91         # converts entire matrix from AMC II to AMC III
pixel by pixel
92         valid_cn_iii = (23 * valid_cn_float) / (10 + 0.13 *
valid_cn_float)
93
94         # calculates composite already on the worst case
scenario
95         cn_iii = np.mean(valid_cn_iii)
96         cn_ii_mean_original = np.mean(valid_cn)
97
98         # SCS CN with saturated soil
99         S = (25400 / cn_iii) - 254
100         Q = np.zeros_like(scenarios)
101
102         valid_p = scenarios > (0.2 * S)
103         Q[valid_p] = ((scenarios[valid_p] - 0.2 * S)**2) / (
scenarios[valid_p] + 0.8 * S)
104
105         volume_rain_m3 = (Q / 1000) * basin_area
106
107         depths = depth_data[mask]

```

```

108         max_depth = depths.max()
109
110         h_range = np.linspace(0, max_depth, 50)
111         v_range = []
112
113         for h in h_range:
114             water_depth = np.maximum(0, h - (max_depth -
115 depths))
116             v_range.append(np.sum(water_depth) * pixel_area)
117
118         final_heights = np.interp(volume_rain_m3, v_range,
119 h_range)
120
121         df = pd.DataFrame({
122             'Basin_ID': int(basin_id),
123             'Precipitation_mm': scenarios,
124             'AMC_II_Original': np.round(cn_ii_mean_original,
125 2),
126             'AMC_III_Saturated': np.round(cn_iii, 2),
127             'Runoff_Q_mm': np.round(Q, 2),
128             'Volume_m3': np.round(volume_rain_m3, 2),
129             'Max_height_m': np.round(final_heights, 2)
130         })
131
132         all_dfs.append(df)
133
134     if all_dfs:
135         combined_df = pd.concat(all_dfs, ignore_index=True)
136         csv_path = os.path.join(out_dir, f'all_basins_{tile_id}
137 _wcs.csv')
138         combined_df.to_csv(csv_path, index=False)
139         print(f"    -> Exported worst-case volume table:
140 all_basins_{tile_id}_wcs.csv")
141
142 def plot_tile_diagnostics(
143     basins_data,
144     transform,
145     core_bounds_5800,
146     buf_deg_bounds,
147     centers,
148     unique_ids,
149     owned_by_core_ids,
150     tile_id,

```

```

147 ):
148     """
149     Takes our results from the dynamic cutting and outputs them
150     as a visual .png file so we can have visual
151     proof that all centroids are contained inside the core box
152     It is advised to read the dynamic_cut function before
153     reading this one (down below).
154     """
155
156     # WARNING: DO NOT REMOVE FIG.
157     fig, ax = plt.subplots(figsize=(12, 10), facecolor='white')
158
159     # 1. Generating unique colors for each ID
160     # Generates random colors for each ID to try to avoid visual
161     # problems with neighbor basins
162     max_id = int(basins_data.max())
163     np.random.seed(42)
164     color_array = np.random.rand(max_id + 1, 4)
165     color_array[0] = [0, 0, 0, 0] # Defines background color (
166     ID 0 = noData)
167     custom_cmap = ListedColormap(color_array)
168
169     # 2. Plots basins raster with custom colors
170     im = ax.imshow(
171         basins_data,
172         cmap=custom_cmap,
173         interpolation='nearest', # Avoids blurring
174         extent=[
175             transform[2],
176             transform[2] + transform[0] * basins_data.shape[1],
177             transform[5] + transform[4] * basins_data.shape[0],
178             transform[5],
179         ],
180     )
181
182     buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
183     df_buf = gpd.GeoDataFrame(
184         {'geometry': [box(buf_minx, buf_miny, buf_maxx, buf_maxy)
185         ]}],
186         crs='EPSG:4326',
187     )
188     b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG:5880').
189     total_bounds
190     c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800

```

```

185
186 # Draws Core Box limits
187 core_rect = patches.Rectangle(
188     (c_minx, c_miny),
189     c_maxx - c_minx,
190     c_maxy - c_miny,
191     linewidth=2.5,
192     edgecolor='red',
193     facecolor='none',
194     linestyle='--',
195     label='Core Tile',
196 )
197 ax.add_patch(core_rect)
198
199 # Draws Core Box + Buffer tile
200 buf_rect = patches.Rectangle(
201     (b_minx, b_miny),
202     b_maxx - b_minx,
203     b_maxy - b_miny,
204     linewidth=2,
205     edgecolor='blue',
206     facecolor='none',
207     label='Buffer Tile (Final)',
208 )
209 ax.add_patch(buf_rect)
210
211 # Plots centroids
212 first_core = True
213 for basin_id, (row_center, col_center) in zip(unique_ids,
214 centers):
215     x_coord, y_coord = transform * (col_center, row_center)
216
217     if basin_id in owned_by_core_ids:
218         label_text = 'Centroid Core' if first_core else ''
219         ax.plot(
220             x_coord,
221             y_coord,
222             marker='o', # <-- Alterado para o ponto preto
223             color='black',
224             markersize=5,
225             markeredgecolor='white',
226             markeredgewidth=0.5,
227             label=label_text,

```

```

227         )
228         first_core = False
229
230     else:
231         ax.plot(
232             x_coord,
233             y_coord,
234             marker='o', # <-- Alterado para o ponto cinza
aparecer!
235             color='dimgray',
236             markersize=2,
237             alpha=0.4,
238         )
239
240     # Ajust zoom
241     ax.set_xlim(b_minx, b_maxx)
242     ax.set_ylim(b_miny, b_maxy)
243
244     # Graphic aesthetics
245     ax.set_facecolor('#f4f4f4')
246     ax.grid(True, linestyle=':', alpha=0.5, color='gray')
247     plt.title(
248         f'Basins check - Tile: {tile_id}',
249         fontsize=14,
250         fontweight='bold',
251     )
252     plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
253     plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
254     plt.legend(loc='upper right', frameon=True, facecolor='white', shadow=True)
255
256     plt.tight_layout()
257     plt.savefig(f'diagnostic_{tile_id}.png', dpi=300)
258     plt.close()
259     print(
260         f'    -> Image stored as: diagnostic_{tile_id}.png'
261     )
262
263 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id):
264     """
265     This will cut the DEM according to topography, preserving
266     basin boundaries (AKA no basins get cut)
267
268     Criteria for cutting: All basins with a centroid that
269     belongs to core tile A will be grouped together on the same

```

```

TIF
267 :param path_vrt: Path to our .vrt file
268 :param out_dir: Path for our output
269 :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
270 :param tile_id: Unique tile ID
271
272 """
273
274 os.makedirs(out_dir, exist_ok=True)
275 minx, miny, maxx, maxy = core_bounds_deg
276
277 # Projects Core Tile limist from 4326 (degress) to 5800 (
meters)
278 df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
maxx, maxy)]}, crs="EPSG:4326")
279 df_core_5800 = df_core.to_crs("EPSG:5880")
280 core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
maxx, maxy) em metros
281
282 # Dynamic buffer initial setup
283 buffer_current = 0.1
284 buffer_steps = 0.1
285 buffer_limit = 4
286 sucess = False
287
288 wbt = WhiteboxTools()
289 wbt.set_working_dir(os.path.abspath(out_dir))
290 wbt.verbose = False
291
292 while not sucess and buffer_current <= buffer_limit:
293     print(f"[{tile_id}] Trying to process with a margin of {
buffer_current}")
294
295     # Define limits
296     buf_minx = minx - buffer_current
297     buf_miny = miny - buffer_current
298     buf_maxx = maxx + buffer_current
299     buf_maxy = maxy + buffer_current
300
301     # Names of temporary files
302     filename_dem = f"temp_{tile_id}_dem.tif"
303     filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
304     filename_d8 = f"temp_{tile_id}_d8.tif"
305     filename_sinks = f"temp_{tile_id}_sinks.tif"

```

```

306     filename_basins = f"basins_final_{tile_id}.tif"
307     filename_depth = f"depth_final_{tile_id}.tif"
308     filename_cn = f"temp_{tile_id}_cn.tif"
309
310     path_dem = os.path.join(out_dir, filename_dem)
311     path_basins = os.path.join(out_dir, filename_basins)
312     path_cn_temp = os.path.join(out_dir, filename_cn)
313
314     try:
315         # Cuts OG VRT
316         gdal.Warp(
317             destNameOrDestDS=path_dem,
318             srcDSOrSrcDSTab=path_vrt,
319             format='GTiff',
320             outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
321             outputBoundsSRS='EPSG:4326',
322             dstSRS='EPSG:5880',
323             xRes=30, yRes=30,
324             creationOptions=['COMPRESS=DEFLATE', 'TILED=YES'
]
325         )
326
327         # Cuts our CN
328         path_cn_original = "BR_CN-2022.tif"
329         gdal.Warp(
330             destNameOrDestDS=path_cn_temp,
331             srcDSOrSrcDSTab=path_cn_original,
332             format='GTiff',
333             outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
334             outputBoundsSRS='EPSG:4326',
335             dstSRS='EPSG:5880',
336             xRes=30, yRes=30,
337             dstNodata=0, # <-- makes sure to have 0's on
noDATA zones
338             creationOptions=['COMPRESS=DEFLATE', 'TILED=YES'
]
339         )
340
341         # WhiteboxTools pipeline here
342         print(f"    -> Running WhiteboxTools ...")
343         wbt.breach_single_cell_pits(dem=filename_dem, output
=filename_dem_clean)

```

```

344         wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
345         wbt.sink(i=filename_dem_clean, output=filename_sinks
, zero_background=True)
346         wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
347
348         # Collision Test pipeline here
349
350         print(f"         -> Running collision test ...")
351
352         with rasterio.open(path_basins) as src:
353             basins_data = src.read(1)
354             transform = src.transform
355
356         # a) Grabs IDS from all basins that touch the borders
of the current matrix (core box + current buffer)
357         top_edge = basins_data[0, :]
358         bottom_edge = basins_data[-1, :]
359         left_edge = basins_data[:, 0]
360         right_edge = basins_data[:, -1]
361
362         edge_ids = set(np.unique(np.concatenate([top_edge,
bottom_edge, left_edge, right_edge])))
363         edge_ids.discard(0) # Removes noData
364
365         # b) Calculates the centroid of each basin present
inside the raster file
366         unique_ids = np.unique(basins_data)
367         unique_ids = unique_ids[unique_ids > 0] # Removes
noData (I hope)
368
369         centers = ndimage.center_of_mass(basins_data > 0,
basins_data, unique_ids)
370
371         owned_by_core_ids = set()
372         c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
373
374         # c) Filters which basins have their centroid INSIDE
core tile
375         for basin_id, (row_center, col_center) in zip(
unique_ids, centers):
376             # Converts pixel (line, column) to geo cords (x,
y in meters)

```

```

377         x_coord, y_coord = transform * (col_center,
row_center)
378
379         if (c_minx <= x_coord <= c_maxx) and (c_miny <=
y_coord <= c_maxy):
380             owned_by_core_ids.add(basin_id)
381
382             # d) Is there any basin with centroid inside core
tile that leaks outside of the buffer?
383             leaking_basins = owned_by_core_ids.intersection(
edge_ids)
384
385             if len(leaking_basins) > 0:
386                 print(f"[FAIL] {len(leaking_basins)} basin(s)
leaked outside current buffer")
387                 buffer_current += buffer_steps
388
389             else:
390                 print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
391
392                 wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
393
394                 # Removing excess basins
395                 print(f"    -> Applying mask to keep only basins
of interest ...")
396
397                 # a) Transforms the valid IDs set into a numpy
array
398                 core_ids_array = np.array(list(owned_by_core_ids
))
399
400                 # b) Creates a boolean mask (True if the pixel
belongs to a basin of interest)
401                 mask = np.isin(basins_data, core_ids_array)
402
403                 # c) Checks which noData value is used by DEM
and applies mask over basin_data
404                 with rasterio.open(path_basins, 'r+') as dst:
405                     lines, columns = basins_data.shape
406                     area_m_square = (lines * 30) * (columns *
30)
407
408                 print(f"    -> Total area processed: {

```

```

area_m_square / 1000000:.2f} km2")
408         nodata_val = dst.nodata if dst.nodata is not
        None else 0
409
410         filtered_basins = np.where(mask, basins_data
, nodata_val)
411         dst.write(filtered_basins, 1)
412
413         # d) Open, filers and overwrites depth TIF
414         path_depth = os.path.join(out_dir,
filename_depth)
415         with rasterio.open(path_depth, 'r+') as dst:
416             depth_data = dst.read(1)
417             nodata_val_depth = dst.nodata if dst.nodata
is not None else 0
418
419             filtered_depth = np.where(mask, depth_data,
nodata_val_depth)
420             dst.write(filtered_depth, 1)
421
422             # testing quick visuals
423             print(f"        -> Generating image ...")
424             buf_bounds = (buf_minx, buf_miny, buf_maxx,
buf_maxy)
425             plot_tile_diagnostics(
426                 filtered_basins,
427                 transform,
428                 core_bounds_5800,
429                 buf_bounds,
430                 centers,
431                 unique_ids,
432                 owned_by_core_ids,
433                 tile_id,
434             )
435
436             # calculating basin geometry and volume / water
height table, alongside some other
437             # important stuff
438
439             print(f"        -> Calculating worst-case
hydrological scenarios\n    Alonside some other important stuff
...")
440
441             shapes_gen = rasterio.features.shapes(

```

```

442         filtered_basins.astype(np.int32),
443         transform=transform
444     )
445
446     records = []
447
448     for geom, value in shapes_gen:
449         if value > 0 and int (value) in
owned_by_core_ids:
450             poly = shape(geom)
451             records.append({
452                 'basin_id': int(value),
453                 'geometry': poly
454             })
455
456         if records:
457             gdf_basins = gpd.GeoDataFrame(records, crs="
EPSG:5880")
458             gdf_basins = gdf_basins.dissolve(by='
basin_id', as_index=False)
459             gdf_basins['area_m2'] = gdf_basins.geometry.
area
460
461             gpkg_path = os.path.join(out_dir, f'
core_basins_{tile_id}.gpkg')
462             gdf_basins.to_file(gpkg_path, driver="GPKG")
463             print(f"        -> Core basin vector stored as:
core_basins_{tile_id}.gpkg")
464
465             simulate_basin_volumes_worst_case(
466                 path_basins=path_basins,
467                 path_depth=path_depth,
468                 path_cn=path_cn_temp,
469                 owned_ids=owned_by_core_ids,
470                 out_dir=out_dir,
471                 tile_id=tile_id
472             )
473             sucess = True
474
475     except Exception as e:
476         print(f"[ERROR]: {e}")
477         print(f"        -> Ending runtime ...")
478         break
479

```

```

480     # Limpeza incluindo o filename_cn!
481     temp_files = [filename_dem, filename_dem_clean,
filename_d8, filename_sinks, filename_cn]
482     if not sucess:
483         temp_files.extend([filename_basins, filename_depth])
# Deletes if failed
484
485     for f in temp_files:
486         path_f = os.path.join(out_dir, f)
487         if os.path.exists(path_f):
488             os.remove(path_f)
489
490     if not sucess and buffer_current >= buffer_limit:
491         print(f"[WARNING]: Buffer limit reached. Consider
revising terrain.")
492
493 if __name__ == "__main__":
494     PATH_VRT = '/home/untitledmacaw/Downloads/output.vrt'
495     OUT_DIR = 'test_basins_dynamic'
496     BOUNDS_TESTING = (-37.7, -9.7, -35.2, -9.2)
497
498     dynamic_cut(
499         path_vrt=PATH_VRT,
500         out_dir=OUT_DIR,
501         core_bounds_deg=BOUNDS_TESTING,
502         tile_id="test_01"
503     )
504

```

For this new code, we get the following results:

```

[test_01] Trying to process with a margin of 0.1
  -> Running WhiteboxTools ...
  -> Running colision test ...
[FAIL] 1 basin(s) leaked outside current buffer
[test_01] Trying to process with a margin of 0.2
  -> Running WhiteboxTools ...
  -> Running colision test ...
[SUCCESS] All interesting basins fit within a buffer of 0.2
  -> Applying mask to keep only basins of interest ...
  -> Total area processed: 38796.10 km2
  -> Generating image ...
  -> Image stored as: diagnostic_test_01.png
  -> Calculating worst-case hydrological scenarios
  Alongside some other important stuff ...
  -> Core basin vector stored as: core_basins_test_01.gpkg
  -> Skipping basin 3: No valid CN data.
  -> Exported worst-case volume table: all_basins_test_01_wcs.csv

```

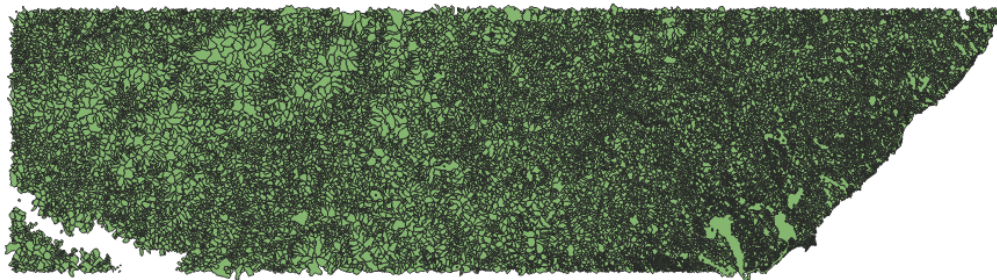
```

▼ test_basins_dynamic
  all_basins_test_01_wcs.csv
  basins_final_test_01.tif
  core_basins_test_01.gpkg
  depth_final_test_01.tif

```

Basin ID	Precipitation_mm	AMC_T1_Original	AMC_T1_Saturated	Runoff_Q_mm	Volume_m3	Max_height_m
2,600.0	89.0	94.9	46.25	5286.13	1.75	
2,180.0	89.0	94.9	164.6	18813.41	2.74	
2,300.0	89.0	94.9	284.22	32486.36	3.36	
2,600.0	89.0	94.9	583.93	66742.73	4.47	
2,800.0	89.0	94.9	783.85	89594.13	5.06	
2,1200.0	89.0	94.9	1183.77	135305.46	5.38	
4,60.0	89.0	94.9	46.25	1831.42	0.96	
4,180.0	89.0	94.9	164.6	6518.03	1.79	
4,300.0	89.0	94.9	284.22	11255.12	2.15	
4,600.0	89.0	94.9	583.93	23123.46	2.35	
4,800.0	89.0	94.9	783.85	31040.49	2.35	
4,1200.0	89.0	94.9	1183.77	46877.48	2.35	
5,60.0	88.65	93.71	43.48	5574820.83	1.44	
5,180.0	88.65	93.71	161.03	20646553.58	1.62	
5,300.0	88.65	93.71	280.46	35958515.92	1.79	
5,600.0	88.65	93.71	580.0	74364558.65	1.9	
5,800.0	88.65	93.71	779.89	99992412.89	1.9	
5,1200.0	88.65	93.71	1179.77	151262899.07	1.9	
6,60.0	89.0	94.9	46.25	10031.16	4.32	
6,180.0	89.0	94.9	164.6	35701.03	6.37	
6,300.0	89.0	94.9	284.22	61647.34	7.19	
6,600.0	89.0	94.9	583.93	126653.52	7.66	
6,800.0	89.0	94.9	783.85	170017.21	7.66	
6,1200.0	89.0	94.9	1183.77	256760.75	7.66	
8,60.0	89.0	94.9	46.25	13902.11	1.54	
8,180.0	89.0	94.9	164.6	49477.77	2.61	
8,300.0	89.0	94.9	284.22	85436.56	3.37	
8,600.0	89.0	94.9	583.93	175528.11	4.61	
8,800.0	89.0	94.9	783.85	235625.51	4.82	
8,1200.0	89.0	94.9	1183.77	355842.69	4.82	
9,60.0	69.65	82.79	23.91	4260.1	0.74	

And for our GeoPackage:



So there you have it! We can:

- Find Bowl-like depressions
- Calculate volume relationships
- Maintain a map of where basins are so we can use that data later with $O(1)$ complexity

But there is still something we are leaving out: We are so far only treating a specific core box that has been chosen by us. We now have to modify our code a little so that we can start handling the entire DEM. But that is already a new section!

5 Applying results for the entire DEM

Alright, we can already see the light at the end of the tunnel! We can already do a lot of fun stuff, but we still have to take all that we've done and use it in our entire dataset (in this case, our downloaded DEM). We need to:

- Unite all of the generated basins GeoPackages so that, in the end, we are left with a single GeoPackage file that will work as a map for all of the basins we've worked with.
- Make adjustments to our CSV table, so that it can store multiple basins instead of just one.
- Define a grid for the original DEM that will consist of multiple different core boxes for us to work with.
- Allow the user to set his/her own paths to files, so that it looks more like a CLI tool and in hopes that it will make FLOWS application in different territory easier
- Add error handling to our code, so that stuff won't explode in case things go south.
- Implement generation of a log file.

We will also make some changes to our previous infiltration function:

- We will change the rainfall accumulation scenarios for calculations sake
- We will also begin to store water height at AMC II also for calculations sake

But, there is still something important that we have to do. I was running the program for the entire ANADEM dataset, and since I wasn't really able to keep the computer running all day all night, I resorted to putting it into suspend mode for the night, and then when I waked up I could resume things as usual. The problem: I don't know, I don't know why, but when the program was on tile 880 of 2820 (our log file was a real life saver here) and already had like a whole week of running, it appears to have kept running even in suspend mode. And, with the strong CPU and RAM usage it was going through, the laptop woke up with a dead battery. That was devastating at first: I was afraid I would need to run the entire thing again. But I had an idea: I already had the compiled files of all 880 previous code boxes, so in theory I could just implement a check in the code that verifies whether we've already processed that basin. The only problem is that we'll have to regenerate all of the lost CSV data, so we'll also implement in our code a function that creates an individual CSV file for each core box, and we'll unite them all in the end.

And with all of that in consideration, our final code looked something like this:

```
1 import os
2 import geopandas as gpd
3 from shapely.geometry import box, shape
4 from osgeo import gdal
5 from whitebox import WhiteboxTools
6 import rasterio
7 import rasterio.features
8 import numpy as np
9 import pandas as pd
10 import scipy.ndimage as ndimage
11 import matplotlib
12 matplotlib.use('Agg') # avoiding some Rich + Tkinter issues
13 import matplotlib.pyplot as plt
14 import matplotlib.patches as patches
15 from matplotlib.colors import ListedColormap
16 from rich.console import Console
17 import time
18 from datetime import datetime
19 from contextlib import contextmanager
20 from pathlib import Path
21
22 console = Console()
23
24 gdal.UseExceptions()
25
26 LOG_FILE = None
27
```

```

28 def log_noprint(*args, **kwargs):
29     """
30     Little brother of log_print function just below this one.
31     Saves status to log without printing it to the user.
32     """
33     if LOG_FILE:
34         try:
35             message = " ".join(str(a) for a in args)
36             with open(LOG_FILE, "a", encoding="utf-8") as f:
37                 timestamp = datetime.now().strftime("%Y-%m-%d %H
38                 :%M:%S")
39                 f.write(f"[{timestamp}] {message}\n")
40         except Exception:
41             pass
42
43 def log_print(*args, **kwargs):
44     """
45     Normal print, but this time it also stores stuff to log!
46     """
47
48     print(*args, **kwargs)
49     if LOG_FILE:
50         try:
51             message = " ".join(str(a) for a in args)
52             with open(LOG_FILE, "a", encoding="utf-8") as f:
53                 timestamp = datetime.now().strftime("%Y-%m-%d %H
54                 :%M:%S")
55                 f.write(f"[{timestamp}] {message}\n")
56         except Exception:
57             pass
58
59 @contextmanager
60 def logged_status(msg, spinner="dots"):
61     """
62     Wrapper for console.status that also saves stuff in log
63     """
64     log_noprint(f"[STATUS] {msg.strip()}")
65     with console.status(msg, spinner="dots") as status:
66         yield status
67
68 def generate_core_boxes(vrt_path, width_deg, height_deg):

```

```

69     """
70     Will take the dimentions of the vrt_path and cut it into
    core boxes for our other functions
71
72     :param vrt_path: Path to VRT file
73     :param width_deg: Desired core box width
74     :param height_deg: Desired core box height
75     """
76     try:
77         with rasterio.open(vrt_path) as src:
78             bounds = src.bounds
79             vrt_minx, vrt_miny, vrt_maxx, vrt_maxy = bounds.left
    , bounds.bottom, bounds.right, bounds.top
80             log_print(f"        -> Total VRT extension:\n    Lon: {
    vrt_minx} - {vrt_maxx}\n    Lat: {vrt_miny} - {vrt_maxy}")
81
82             boxes = []
83
84             x_coords = np.arange(vrt_minx, vrt_maxx, width_deg)
85             y_coords = np.arange(vrt_miny, vrt_maxy, height_deg)
86
87             tile_count = 1
88
89             for y in y_coords:
90                 for x in x_coords:
91                     box_maxx = min(x + width_deg, vrt_maxx)
92                     box_maxy = min(y + height_deg, vrt_maxy)
93
94                     bbox = (x, y, box_maxx, box_maxy)
95
96                     boxes.append({
97                         "tile_id": f"tile_{tile_count:04d}",
98                         "bounds": bbox
99                     })
100
101                     tile_count += 1
102
103             log_print(f"        -> Total of generated core boxes: {len(
    boxes)}")
104
105             return boxes
106         except Exception as e:
107             raise RuntimeError(f"Error while generating core boxes:
    {e}")

```

```

108
109 def simulate_basin_volumes_worst_case(path_basins, path_depth,
    path_cn, owned_ids, out_dir, tile_id):
110     """
111     Analyses individual basins, and for each one calculates:
112     -> Composite CN (after we convert each one from AMC II to
    AMC III)
113     -> Volume / Water height relations
114     -> Other important stuff
115
116     and then stores the values in a cute table!
117
118     :param path_basins: Path to our basins_final .tif file
119     :param path_depth: Path to our depth_final .tif file
120     :param path_cn: Path to our CN table .tif file (observation:
    their CN values are AMC II standard. We must convert them
    first)
121     :param owned_ids: Our owned_by_core_ids variable (go to the
    main function - dynamic_cut - if in doubt)
122     :param out_dir: Our output directory
123     :param tile_id: Our tile id (go to the main function if in
    doubt)
124
125     """
126     try:
127         # saves different storm strenghts to test (in mm)
128         # according to opticalscientific.com:
129         # light rain: < 60mm / 24hrs ( < 2.5mm / hour, and we
    converted it to 24hrs)
130         # moderate rain: 60 to 180mm / 24hrs (2.5 to 7.5 mm /
    hour)
131         # heavy rain: 180mm to 1200mm / 24hrs (7.5 to 50mm /
    hour)
132         # Violent rain: > 1200 / 24hrs ( > 50mm / hour)
133         # here, we will add some in-between values for
    calculations' sake
134         scenarios = np.array([30, 60, 100, 150, 250, 400, 700],
    dtype=float)
135
136         all_dfs = []
137
138         with rasterio.open(path_basins) as src_b, \
139             rasterio.open(path_depth) as src_d, \
140             rasterio.open(path_cn) as src_cn:

```

```

141
142     # Validates dimentions (between our CN and DEM)
143     if not (src_b.shape == src_d.shape == src_cn.shape):
144         raise ValueError(
145             f"[ALIGNMENT ERROR] Dimentions are not
compatible\n"
146             f"         -> Basins: {src_b.shape}\n"
147             f"         -> Depth:  {src_d.shape}\n"
148             f"         -> CN:      {src_cn.shape}"
149         )
150
151     # 2. Validates resolution and geographical
positioning (Geotransform)
152     # Allows a tiny tiny tolerance for floating point
Warp imperfections
153     if not np.allclose(src_b.transform, src_cn.transform
, atol=1e-5):
154         raise ValueError(
155             "[ALIGNMENT ERROR] The matrixes cover
entirely different areas or have different spacial resolution
"
156         )
157
158     basins_data = src_b.read(1)
159     depth_data = src_d.read(1)
160     cn_data = src_cn.read(1)
161
162     pixel_width = abs(src_b.transform[0])
163     pixel_height = abs(src_b.transform[4])
164     pixel_area = pixel_width * pixel_height
165
166     for basin_id in owned_ids:
167         mask = (basins_data == basin_id)
168         if not np.any(mask):
169             continue
170
171         basin_area = np.sum(mask) * pixel_area
172
173         valid_cn = cn_data[mask]
174         valid_cn = valid_cn[valid_cn > 0]
175         # Converts array to float to prevent uint8
overflow
176
177         if len(valid_cn) == 0:

```

```

178         log_print(f"        -> Skipping basin {basin_id
}: No valid CN data.")
179         continue
180
181         valid_cn_float = valid_cn.astype(float)
182
183         # converts entire matrix from AMC II to AMC III
pixel by pixel
184         valid_cn_iii = (23 * valid_cn_float) / (10 +
0.13 * valid_cn_float)
185
186         # calculates composite already on the worst case
scenario
187         cn_iii = np.mean(valid_cn_iii)
188         cn_ii_mean_original = np.mean(valid_cn)
189
190         # SCS CN with saturated soil
191         S_iii = (25400 / cn_iii) - 254
192         Q_iii = np.zeros_like(scenarios)
193
194         valid_p_iii = scenarios > (0.2 * S_iii)
195         Q_iii[valid_p_iii] = ((scenarios[valid_p_iii] -
0.2 * S_iii)**2) / (scenarios[valid_p_iii] + 0.8 * S_iii)
196
197         volume_rain_m3_iii = (Q_iii / 1000) * basin_area
198
199         # SCS CN on regular moisture conditions
200         S_ii = (25400 / cn_ii_mean_original) - 254
201         Q_ii = np.zeros_like(scenarios)
202
203         valid_p_ii = scenarios > (0.2 * S_ii)
204         Q_ii[valid_p_ii] = ((scenarios[valid_p_ii] - 0.2
* S_ii)**2 ) / (scenarios[valid_p_ii] + 0.8 * S_ii)
205
206         volume_rain_m3_ii = (Q_ii / 1000) * basin_area
207
208         #height interpolation
209
210         depths = depth_data[mask]
211         max_depth = depths.max()
212
213         h_range = np.linspace(0, max_depth, 50)
214         v_range = []
215

```

```

216         for h in h_range:
217             water_depth = np.maximum(0, h - (max_depth -
depths))
218             v_range.append(np.sum(water_depth) *
pixel_area)
219
220             final_heights_iii = np.interp(volume_rain_m3_iii
, v_range, h_range)
221             final_heights_ii = np.interp(volume_rain_m3_ii,
v_range, h_range)
222
223             #exporting to final file
224
225             df = pd.DataFrame({
226                 'Tile ID': tile_id,
227                 'Basin ID': int(basin_id),
228                 'Precipitation_mm': scenarios,
229                 'AMC_II_Original': np.round(
cn_ii_mean_original, 2),
230                 'AMC_III_Saturated': np.round(cn_iii, 2),
231                 'Runoff_Q_mm_AMC_II': np.round(Q_ii, 2),
232                 'Volume_m3_AMC_II': np.round(
volume_rain_m3_ii, 2),
233                 'Max_height_m_AMC_II': np.round(
final_heights_ii, 2),
234                 'Runoff_Q_mm_AMC_III': np.round(Q_iii, 2),
235                 'Volume_m3_AMC_III': np.round(
volume_rain_m3_iii, 2),
236                 'Max_height_m_AMC_III': np.round(
final_heights_iii, 2)
237             })
238
239             all_dfs.append(df)
240
241         if all_dfs:
242             combined_df = pd.concat(all_dfs, ignore_index=True)
243             return combined_df
244
245         return None # returns nothing if we don't find anything
246     except Exception as e:
247         raise RuntimeError(f"Error on basin rainfall simulaton:
{e}")
248
249

```

```

250 def plot_tile_diagnostics(
251     basins_data,
252     transform,
253     core_bounds_5800,
254     buf_deg_bounds,
255     centers,
256     unique_ids,
257     owned_by_core_ids,
258     tile_id,
259     out_dir
260 ):
261     """
262     Takes our results from the dynamic cutting and outputs them
263     as a visual .png file so we can have visual
264     proof that all centroids are contained inside the core box
265     It is advised to read the dynamic_cut function before
266     reading this one (down below).
267     """
268     try:
269         # WARNING: DO NOT REMOVE FIG.
270         fig, ax = plt.subplots(figsize=(12, 10), facecolor='
white')
271
272         # 1. Generating unique colors for each ID
273         # Generates random colors for each ID to try to avoid
274         visual problems with neighbor basins
275         max_id = int(basins_data.max())
276         np.random.seed(42)
277         color_array = np.random.rand(max_id + 1, 4)
278         color_array[0] = [0, 0, 0, 0] # Defines background
279         color (ID 0 = noData)
280         custom_cmap = ListedColormap(color_array)
281
282         # 2. Plots basins raster with custom colors
283         im = ax.imshow(
284             basins_data,
285             cmap=custom_cmap,
286             interpolation='nearest', # Avoids blurring
287             extent=[
288                 transform[2],
289                 transform[2] + transform[0] * basins_data.shape
290 [1],
291                 transform[5] + transform[4] * basins_data.shape
292 [0],

```

```

287         transform[5],
288     ],
289 )
290
291     buf_minx, buf_miny, buf_maxx, buf_maxy = buf_deg_bounds
292     df_buf = gpd.GeoDataFrame(
293         {'geometry': [box(buf_minx, buf_miny, buf_maxx,
294             buf_maxy)]},
295         crs='EPSG:4326',
296     )
297     b_minx, b_miny, b_maxx, b_maxy = df_buf.to_crs('EPSG
298 :5880').total_bounds
299     c_minx, c_miny, c_maxx, c_maxy = core_bounds_5800
300
301     # Draws Core Box limits
302     core_rect = patches.Rectangle(
303         (c_minx, c_miny),
304         c_maxx - c_minx,
305         c_maxy - c_miny,
306         linewidth=2.5,
307         edgecolor='red',
308         facecolor='none',
309         linestyle='--',
310         label='Core Tile',
311     )
312     ax.add_patch(core_rect)
313
314     # Draws Core Box + Buffer tile
315     buf_rect = patches.Rectangle(
316         (b_minx, b_miny),
317         b_maxx - b_minx,
318         b_maxy - b_miny,
319         linewidth=2,
320         edgecolor='blue',
321         facecolor='none',
322         label='Buffer Tile (Final)',
323     )
324     ax.add_patch(buf_rect)
325
326     # Plots centroids
327     first_core = True
328     for basin_id, (row_center, col_center) in zip(unique_ids
329         , centers):
330         x_coord, y_coord = transform * (col_center,

```

```

row_center)

328
329         if basin_id in owned_by_core_ids:
330             label_text = 'Centroid Core' if first_core else
331             ''
332             ax.plot(
333                 x_coord,
334                 y_coord,
335                 marker='o', # <-- Alterado para o ponto
336                 preto aparecer!
337                 color='black',
338                 markersize=5,
339                 markeredgecolor='white',
340                 markeredgewidth=0.5,
341                 label=label_text,
342             )
343             first_core = False
344
345         else:
346             ax.plot(
347                 x_coord,
348                 y_coord,
349                 marker='o', # <-- Alterado para o ponto
350                 cinza aparecer!
351                 color='dimgray',
352                 markersize=2,
353                 alpha=0.4,
354             )
355
356         # Ajust zoom
357         ax.set_xlim(b_minx, b_maxx)
358         ax.set_ylim(b_miny, b_maxy)
359
360         # Graphic aesthetics
361         ax.set_facecolor('#f4f4f4')
362         ax.grid(True, linestyle=':', alpha=0.5, color='gray')
363         plt.title(
364             f'Basins check - Tile: {tile_id}',
365             fontsize=14,
366             fontweight='bold',
367         )
368         plt.xlabel('Easting (Meters - EPSG:5880)', fontsize=11)
369         plt.ylabel('Northing (Meters - EPSG:5880)', fontsize=11)
370         plt.legend(loc='upper right', frameon=True, facecolor='

```

```

white', shadow=True)
368
369     plt.tight_layout()
370     img_path = os.path.join(out_dir, f'diagnostic_{tile_id}.
png')
371     plt.savefig(img_path, dpi=300)
372     plt.close()
373     log_print(
374         f'        -> Image stored as: diagnostic_{tile_id}.png'
375     )
376
377     except Exception as e:
378         raise RuntimeError(f"Erro while plotting image
diagnostics: {e}")
379
380 def dynamic_cut(path_vrt, out_dir, core_bounds_deg, tile_id,
path_cn_input):
381     """
382     This will cut the DEM according to topography, preserving
basin boundaries (AKA no basins get cut)
383     Criteria for cutting: All basins with a centroid that
belongs to core tile A will be grouped together on the same
TIF
384     :param path_vrt: Path to our .vrt file
385     :param out_dir: Path for our output
386     :param code_bounds_deg: Tuple (min_x, min_y, max_x, max_y)
387     :param tile_id: Unique tile ID
388
389     """
390
391     os.makedirs(out_dir, exist_ok=True)
392     minx, miny, maxx, maxy = core_bounds_deg
393
394     path_csv_file = Path(out_dir) / f"flows_table_{tile_id}.csv"
395
396     # IMPORTANT -> checks if this code box was already compiled
397
398     if path_csv_file.is_file():
399         log_print(f"[NOTE] {tile_id} seems to already be 100%
processed. Moving on to the next one")
400         return None
401
402
403     df_tile = None

```

```

404
405     # Projects Core Tile limist from 4326 (degress) to 5800 (
meters)
406     df_core = gpd.GeoDataFrame({'geometry': [box(minx, miny,
maxx, maxy)]}, crs="EPSG:4326")
407     df_core_5800 = df_core.to_crs("EPSG:5880")
408     core_bounds_5800 = df_core_5800.total_bounds # (minx, miny,
maxx, maxy) em metros
409
410     # Dynamic buffer initial setup
411     buffer_current = 0.1
412     buffer_steps = 0.1
413     buffer_limit = 4
414     sucess = False
415
416     wbt = WhiteboxTools()
417     wbt.set_working_dir(os.path.abspath(out_dir))
418     wbt.verbose = False
419
420     while not sucess and buffer_current <= buffer_limit:
421         log_print(f"[{tile_id}] Trying to process with a margin
of {buffer_current}")
422
423         # Define limits
424         buf_minx = minx - buffer_current
425         buf_miny = miny - buffer_current
426         buf_maxx = maxx + buffer_current
427         buf_maxy = maxy + buffer_current
428
429         # Names of temporary files
430         filename_dem = f"temp_{tile_id}_dem.tif"
431         filename_dem_clean = f"temp_{tile_id}_dem_clean.tif"
432         filename_d8 = f"temp_{tile_id}_d8.tif"
433         filename_sinks = f"temp_{tile_id}_sinks.tif"
434         filename_basins = f"basins_final_{tile_id}.tif"
435         filename_depth = f"depth_final_{tile_id}.tif"
436         filename_cn = f"temp_{tile_id}_cn.tif"
437
438         path_dem = os.path.join(out_dir, filename_dem)
439         path_basins = os.path.join(out_dir, filename_basins)
440         path_cn_temp = os.path.join(out_dir, filename_cn)
441
442         try:
443             # Cuts OG VRT

```

```

444         gdal.Warp(
445             destNameOrDestDS=path_dem,
446             srcDSOrSrcDSTab=path_vrt,
447             format='GTiff',
448             outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
449             outputBoundsSRS='EPSG:4326',
450             dstSRS='EPSG:5880',
451             xRes=30, yRes=30,
452             creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]
453         )
454
455         # Cuts our CN
456         path_cn_original = path_cn_input
457         gdal.Warp(
458             destNameOrDestDS=path_cn_temp,
459             srcDSOrSrcDSTab=path_cn_original,
460             format='GTiff',
461             outputBounds=(buf_minx, buf_miny, buf_maxx,
buf_maxy),
462             outputBoundsSRS='EPSG:4326',
463             dstSRS='EPSG:5880',
464             xRes=30, yRes=30,
465             dstNodata=0, # <-- makes sure to have 0's on
noDATA zones
466             creationOptions=['COMPRESS=DEFLATE', 'TILED=YES '
]
467         )
468
469         # WhiteboxTools pipeline here
470         with logged_status("Running WhiteboxTools ", spinner
="dots"):
471             wbt.breach_single_cell_pits(dem=filename_dem,
output=filename_dem_clean)
472             wbt.d8_pointer(dem=filename_dem_clean, output=
filename_d8)
473             wbt.sink(i=filename_dem_clean, output=
filename_sinks, zero_background=True)
474             wbt.watershed(d8_pntr=filename_d8, pour_pts=
filename_sinks, output=filename_basins)
475
476
477         # Colision Test pipeline here

```

```

478         with logged_status("Running colision test ", spinner
479         ="dots"):
480             with rasterio.open(path_basins) as src:
481                 basins_data = src.read(1)
482                 transform = src.transform
483
484                 # a) Grabs IDS from all basins that touch the
485                 bordes of the current matrix (core box + current buffer)
486                 top_edge = basins_data[0, :]
487                 bottom_edge = basins_data[-1, :]
488                 left_edge = basins_data[:, 0]
489                 right_edge = basins_data[:, -1]
490
491                 edge_ids = set(np.unique(np.concatenate([
492                 top_edge, bottom_edge, left_edge, right_edge])))
493                 edge_ids.discard(0) # Removes noData
494
495                 # b) Calculates the centroid of each basin
496                 present inside the raster file
497                 unique_ids = np.unique(basins_data)
498                 unique_ids = unique_ids[unique_ids > 0] #
499                 Removes noData (I hope)
500
501                 centers = ndimage.center_of_mass(basins_data >
502                 0, basins_data, unique_ids)
503
504                 owned_by_core_ids = set()
505                 c_minx, c_miny, c_maxx, c_maxy =
506                 core_bounds_5800
507
508                 # c) Filters which basins have their centroid
509                 INSIDE core tile
510                 for basin_id, (row_center, col_center) in zip(
511                 unique_ids, centers):
512                     # Converts pixel (line, column) to geo cords
513                     (x, y in meters)
514                     x_coord, y_coord = transform * (col_center,
515                     row_center)
516
517                     if (c_minx <= x_coord <= c_maxx) and (c_miny
518                     <= y_coord <= c_maxy):
519                         owned_by_core_ids.add(basin_id)
520
521                 # c.1) Some basins may be basically just ocean,

```

```

thus won't contain
510         # any basins. Let's skip the core boxes where
that happens
511         if len(owned_by_core_ids) == 0:
512             log_print(f"[SKIP] No valid basins found (
likely pure ocean or Nodata).")
513
514             temp_files = [filename_dem,
filename_dem_clean, filename_d8, filename_sinks, filename_cn,
filename_basins]
515             for f in temp_files:
516                 path_f = os.path.join(out_dir, f)
517                 if os.path.exists(path_f):
518                     os.remove(path_f)
519
520             return None
521
522             # d) Is there any basin with centroid inside
core tile that leaks outside of the buffer?
523             leaking_basins = owned_by_core_ids.intersection(
edge_ids)
524
525             if len(leaking_basins) > 0:
526                 log_print(f"[FAIL] {len(leaking_basins)} basin(s
) leaked outside current buffer")
527                 buffer_current += buffer_steps
528
529             else:
530                 log_print(f"[SUCESS] All interesting basins fit
within a buffer of {buffer_current}")
531
532                 wbt.depth_in_sink(dem=filename_dem_clean, output
=filename_depth, zero_background=True)
533
534                 # Removing excess basins
535                 with logged_status("Applying mask to keep only
basins of interest ", spinner="dots"):
536
537                     # a) Transforms the valid IDs set into a
numpy array
538                     core_ids_array = np.array(list(
owned_by_core_ids))
539
540                     # b) Creates a boolean mask (True if the

```

```

pixel belongs to a basin of interest)
541         mask = np.isin(basins_data, core_ids_array)
542
543         # c) Checks which noData value is used by
DEM and applies mask over basin_data
544         with rasterio.open(path_basins, 'r+') as dst
:
545             lines, columns = basins_data.shape
546             area_m_square = (lines * 30) * (columns
* 30)
547             log_print(f"        -> Total area processed:
{area_m_square / 1000000:.2f} km2")
548             nodata_val = dst.nodata if dst.nodata is
not None else 0
549
550             filtered_basins = np.where(mask,
basins_data, nodata_val)
551             dst.write(filtered_basins, 1)
552
553             # d) Open, filters and overwrites depth TIF
554             path_depth = os.path.join(out_dir,
filename_depth)
555             with rasterio.open(path_depth, 'r+') as dst:
556                 depth_data = dst.read(1)
557                 nodata_val_depth = dst.nodata if dst.
nodata is not None else 0
558
559                 filtered_depth = np.where(mask,
depth_data, nodata_val_depth)
560                 dst.write(filtered_depth, 1)
561
562             log_print(f"        -> Mask applied!")
563             # testing quick visuals
564             with logged_status("Generating image ", spinner=
"dots"):
565                 buf_bounds = (buf_minx, buf_miny, buf_maxx,
buf_maxy)
566                 plot_tile_diagnostics(
567                     filtered_basins,
568                     transform,
569                     core_bounds_5800,
570                     buf_bounds,
571                     centers,
572                     unique_ids,

```

```

573         owned_by_core_ids,
574         tile_id,
575         out_dir
576     )
577
578     # calculating basin geometry and volume / water
height table, alongside some other
579     # important stuff
580     with logged_status("Calculating worst-case
hydrological scenarios\n Alongside some other important stuff
", spinner="dots"):
581         shapes_gen = rasterio.features.shapes(
582             filtered_basins.astype(np.int32),
583             transform=transform
584         )
585
586         records = []
587
588         for geom, value in shapes_gen:
589             if value > 0 and int (value) in
owned_by_core_ids:
590                 poly = shape(geom)
591                 records.append({
592                     'basin_id': int(value),
593                     'geometry': poly
594                 })
595
596             if records:
597                 gdf_basins = gpd.GeoDataFrame(records,
crs="EPSG:5880")
598                 gdf_basins = gdf_basins.dissolve(by='
basin_id', as_index=False)
599                 gdf_basins['area_m2'] = gdf_basins.
geometry.area
600                 gdf_basins['Tile_ID'] = tile_id
601
602                 gpkg_path = os.path.join(out_dir, f'
core_basins_{tile_id}.gpkg')
603                 gdf_basins.to_file(gpkg_path, driver="
GPKG")
604                 log_print(f"         -> Core basin vector
stored as: core_basins_{tile_id}.gpkg")
605
606                 df_tile = simulate_basin_volumes_worst_case(

```

```

607         path_basins=path_basins,
608         path_depth=path_depth,
609         path_cn=path_cn_temp,
610         owned_ids=owned_by_core_ids,
611         out_dir=out_dir,
612         tile_id=tile_id
613     )
614
615     # Converting individual core data to CSV
616
617     if df_tile is not None and not df_tile.empty
618 :
619         df_tile.to_csv(path_csv_file, index=
False)
620
621         log_print(f"        -> Regenerated CSV table
as: flows_table_{tile_id}.csv")
622
623         sucess = True
624
625     except Exception as e:
626         raise RuntimeError(f"Critical Failure on {tile_id}:
{e}")
627
628     finally:
629         # Limpeza incluindo o filename_cn!
630         temp_files = [filename_dem, filename_dem_clean,
filename_d8, filename_sinks, filename_cn, filename_basins,
filename_depth]
631
632         for f in temp_files:
633             path_f = os.path.join(out_dir, f)
634             if os.path.exists(path_f):
635                 os.remove(path_f)
636
637         if not sucess and buffer_current >= buffer_limit:
638             raise RuntimeError(f"Buffer limit reached on {tile_id}.
Consider revising terrain.")
639
640         return None
641
642 if __name__ == "__main__":
643     log_print("Hello! Welcome to FLOWS command tool")
644     time.sleep(0.5)

```

```

644     PATH_VRT = str(input("Path for VRT: ")).strip('\n')
645     PATH_CN_INPUT = str(input("Path for CN raster: ")).strip('\n')
646     OUT_DIR = str(input("Output folder name (e.g., output_dir): ")).strip('\n')
647
648     TILE_WIDTH = float(input("Tile width in degrees (e.g., 2.5): "))
649     TILE_HEIGHT = float(input("Tile height in degrees (e.g., 0.5): "))
650
651     os.makedirs(OUT_DIR, exist_ok=True)
652     LOG_FILE = os.path.join(OUT_DIR, "log.txt")
653
654     time.sleep(0.5)
655     log_print(f"Log will be saved in {OUT_DIR}/log.txt")
656     time.sleep(0.5)
657
658     start_time = time.time()
659
660     with logged_status("Generating grid", spinner="dots"):
661         all_core_boxes = generate_core_boxes(PATH_VRT,
662                                           TILE_WIDTH, TILE_HEIGHT)
663
664         for box_info in all_core_boxes:
665             tile_name = box_info["tile_id"]
666             tile_bounds = box_info["bounds"]
667
668             try:
669                 with logged_status(f"Processing {tile_name} | Bounds : {tile_bounds}"):
670                     dynamic_cut(
671                         path_vrt=PATH_VRT,
672                         out_dir=OUT_DIR,
673                         core_bounds_deg=tile_bounds,
674                         tile_id=tile_name,
675                         path_cn_input=PATH_CN_INPUT
676                     )
677
678             except Exception as e:
679                 log_print(f"[ERROR]: {e}")
680                 time.sleep(0.5)
681                 log_print(f"    -> Aborting all subsequent tiles")
682                 break

```

```

682
683     with logged_status("Uniting CSV files", spinner="dots"):
684         csv_files = [
685             os.path.join(OUT_DIR, f) for f in os.listdir(OUT_DIR
686 ) if f.startswith("flows_table_tile") and f.endswith(".csv")
687         ]
688
689     if csv_files:
690         df_list = [pd.read_csv(f) for f in csv_files]
691         final_df = pd.concat(df_list, ignore_index=True)
692
693         final_csv_path = os.path.join(OUT_DIR, '
694 FLOWS_all_basins.csv')
695         final_df.to_csv(final_csv_path, index=False)
696
697         log_print(f"[SUCCESS] Final table stored on {
698 final_csv_path}")
699         log_print(f"    -> Total of lines: {len(final_df)}")
700
701         # Limpando paradinhas aqui
702         for stuff in csv_files:
703             os.remove(stuff)
704
705     else:
706         log_print("[WARNING] No csv data found")
707
708     with logged_status("Generating main GeoPackage and cleaning
709 auxiliaries"):
710         gpkg_files = [
711             os.path.join(OUT_DIR, f)
712             for f in os.listdir(OUT_DIR)
713             if f.startswith("core_basins_tile_") and f.
714 endswith(".gpkg")
715         ]
716
717         if gpkg_files:
718             gdf_list = [gpd.read_file(f) for f in gpkg_files
719 ]
720
721             merge_gdf = gpd.GeoDataFrame(pd.concat(gdf_list,
722 ignore_index=True), crs=gdf_list[0].crs)
723
724             final_gpkg_path = os.path.join(OUT_DIR, '
725 FLOWS_basins_map.gpkg')

```

```

718         merge_gdf.to_file(final_gpkg_path, driver="GPKG"
)
719         log_print(f"        -> Merged map saved as
FLOWS_basins_map.gpkg")
720
721         # cleaning up temp files
722         for f in gpkg_files:
723             os.remove(f)
724
725         log_print(f"        -> Temporary files removed
sucessfully")
726
727         end_time = time.time()
728         elapsed_time = end_time - start_time
729         mins = int(elapsed_time // 60)
730         secs = int(elapsed_time % 60)
731         log_print("[END OF RUNTIME] Program finished")
732         log_print(f"Total elapsed time: {mins}m {secs}s")
733

```

Now take in consideration that this code might take a while to run. In my case, I couldn't really leave it running nonstop all day and night, so I was getting a max of 8 hours of runtime daily. Progress in the first days looked something like this:

- Day 1 → About 200 / 2820
- Day 2 → About 280 / 2820
- Day 3 → About 305 / 2820
- Day 1 → About 424 / 2820

NOTE Currently in progress. Compiling hasn't finished yet, therefore this documentation is not finished yet